

SK가스 임직원 · 1Day

# 바이브코딩 Oto1



게임 체험

LV1-2 · 만들고 한 가지 고치기



내 웹앱

LV3-5 · 인터뷰부터 직접 확인까지



업무 도구

LV6-7 · CSV 분석 → Work OS

# 강사 박성훈 소개

AI를 실제 업무에 적용하는 방법을 함께 찾습니다.

## 이력과 현재 하는 일

- 컬럼비아대학교 컴퓨터공학 전공 · 휴학
- 스타트업 창업
- 현재 개인·기업 대상

**AX 코칭 · 컨설팅 · 교육 진행**



# 오늘 가져갈 결과물과 사용할 도구

## 오늘 가져갈 결과물

- 직접 만든 게임 · OX 퀴즈 · 사진 기록 웹앱
- CSV 분석 서비스와  
메모·할 일을 더한 Work OS
- 내 아이디어의 기획 문서와 프로젝트 파일

## 오늘 사용할 도구

### Gemini

게임을 만들고 실행해 보기

### VS Code

프로젝트 파일을 보고 작업하기

### Claude Code

인터뷰 · 기획 · 구현 · 수정

전체 흐름

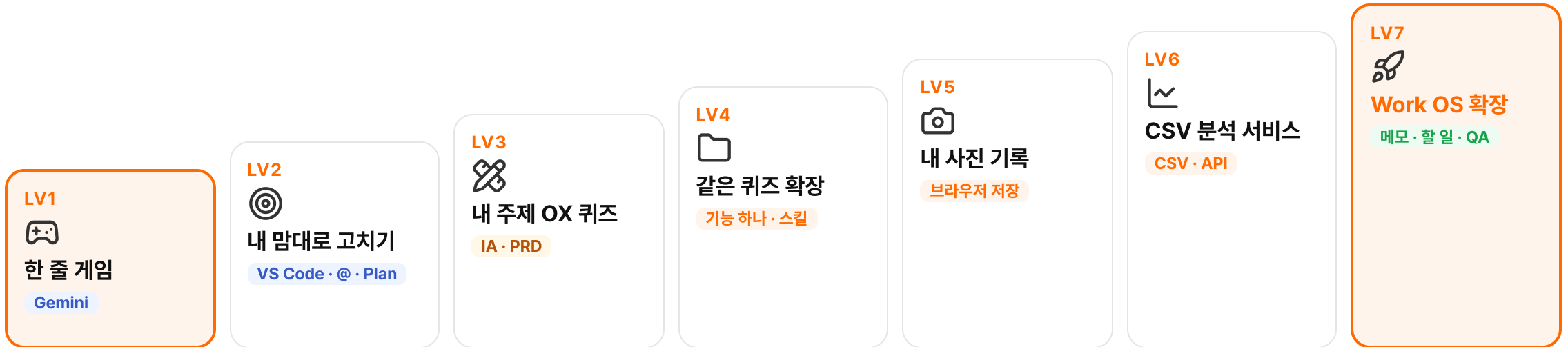
## 일곱 레벨의 학습 흐름



게임은 체험, 퀴즈는 첫 정식 루프. CSV 서비스는 Work OS로 이어집니다.

전체 흐름

## 일곱 레벨의 학습 흐름

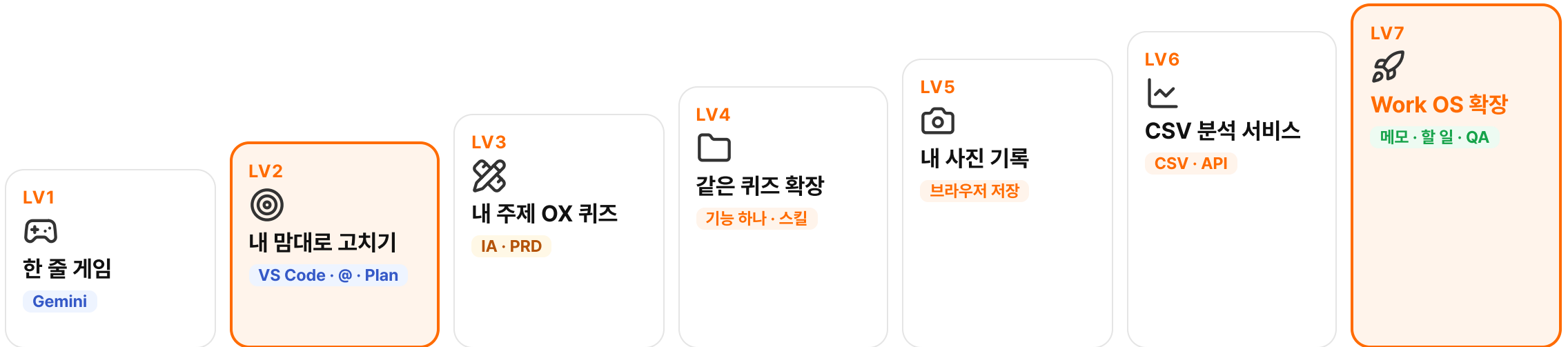


### LV1 · 한 줄 게임 ★★★★★

두 디자인 프롬프트를 별도 새 채팅에 넣어 비교합니다. 하나를 골라 실행하고 바꾸고 싶은 점을 찾습니다. · Gemini 웹

전체 흐름

# 일곱 레벨의 학습 흐름



## LV2 · 내 맘대로 고치기 ★★★★★

Gemini에서 만든 HTML을 가져와 변화 하나를 구현합니다. 프롬프트를 AI와 함께 정하고 직접 확인합니다. · VS Code + Claude Code

전체 흐름

## 일곱 레벨의 학습 흐름

LV1



한 줄 게임

Gemini

LV2



내 맘대로 고치기

VS Code · @ · Plan

LV3



내 주제 OX 퀴즈

IA · PRD

LV4



같은 퀴즈 확장

기능 하나 · 스킬

LV5



내 사진 기록

브라우저 저장

LV6



CSV 분석 서비스

CSV · API

LV7



Work OS 확장

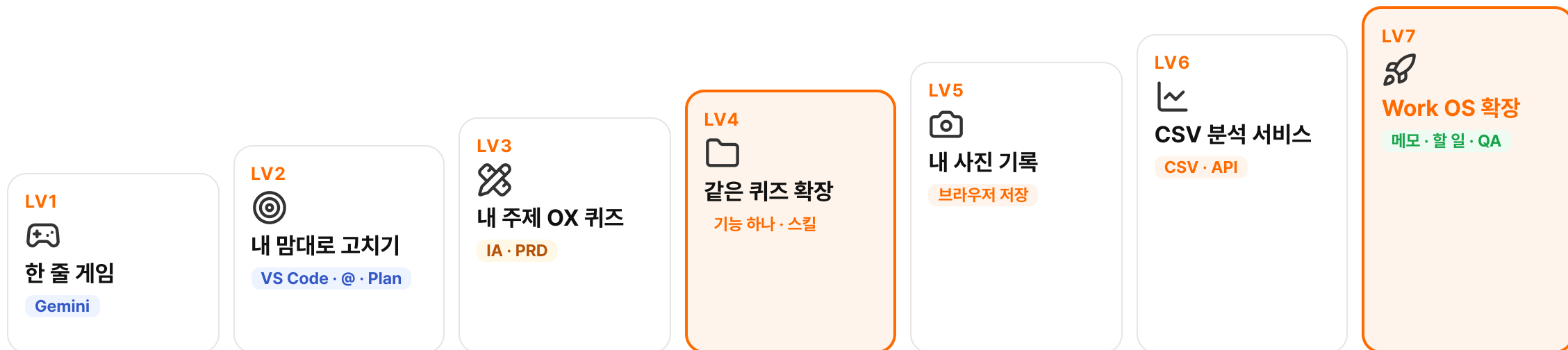
메모 · 할 일 · QA

LV3 · 내 주제 OX 퀴즈 ★★☆☆☆

아이디어·구현 문서·피드백을 정하고 OX 퀴즈를 구현·검증합니다. · VS Code + Claude Code

전체 흐름

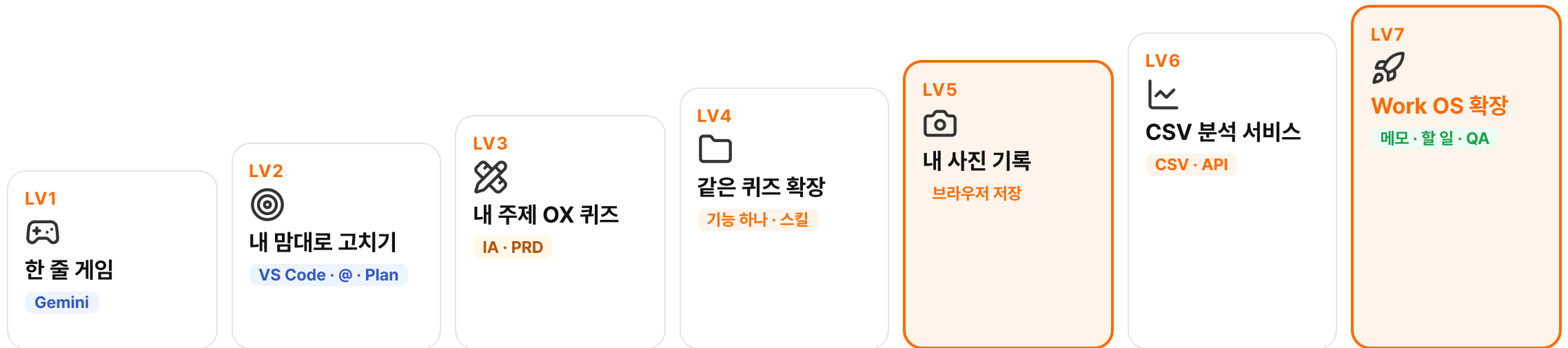
## 일곱 레벨의 학습 흐름



### LV4 · 같은 퀴즈에 기능 하나 ★★☆☆☆

같은 퀴즈의 구현 문서를 고치고 기능 하나를 개선·검증합니다. · VS Code + Claude Code

# 일곱 레벨의 학습 흐름



## LV5 · 내 사진 기록 ★★★★★

grill-me(기획 질문 도우미)로 빠진 조건을 찾습니다. 사진·메모를 저장하고 나중에 다시 봅니다. API를 더하는 방법은 선택 자료에서 봅니다.

전체 흐름

# 일곱 레벨의 학습 흐름

LV1



한 줄 게임

Gemini

LV2



내 맘대로 고치기

VS Code · @ · Plan

LV3



내 주제 OX 퀴즈

IA · PRD

LV4



같은 퀴즈 확장

기능 하나 · 스킬

LV5



내 사진 기록

브라우저 저장

LV6



CSV 분석 서비스

CSV · API

LV7



Work OS 확장

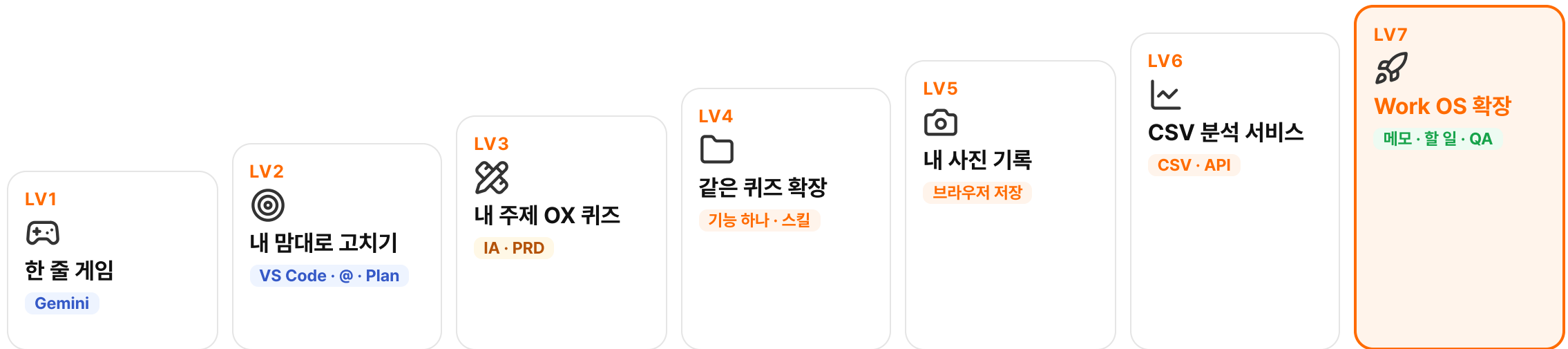
메모 · 할 일 · QA

## LV6 · CSV 분석 서비스 ★★☆☆☆

가상 CSV를 직접 만들고 Claude가 분석해주는 서비스를 기획·제작합니다. 분석 질문과 결과 화면은 인터뷰로 정합니다.

전체 흐름

## 일곱 레벨의 학습 흐름



### LV7 · Work OS 확장·검증 ★★★★★

LV6의 CSV 분석 서비스에 메모·할 일 관리 기능을 더해 Work OS로 확장합니다. QA로 확인하고 압축해 전달합니다.

# 작품이 달라도 **같은 순서**



↪ 기대와 실제의 차이를 다음 수정으로

퀴즈에서 한 번 익히고 계속 사용합니다

게임은 가볍게 체험합니다.

퀴즈 → 사진 → CSV 분석 → Work OS에서도 같은 순서로 만듭니다.

# 하루 시간표와 휴식

| 시간    | 레벨                                                | 난이도   |
|-------|---------------------------------------------------|-------|
| 09:00 | 시작 전체 흐름 · 환경 점검 → 개념과 트렌드                        |       |
| 09:55 | LV1-2 게임 생성 · Gemini → 수정 · VS Code + Claude Code | ★☆☆☆☆ |
| 10:50 | 휴식                                                |       |
| 11:05 | LV3 내 주제 OX 퀴즈 · VS Code + Claude Code            | ★★☆☆☆ |
| 12:00 | 점심                                                |       |
| 13:00 | LV4-5 계획 기반 반복 개발 → 내 사진 기록 · VS Code             | ★★★☆☆ |
| 14:50 | 휴식                                                |       |
| 15:05 | LV6-7① CSV 분석 서비스 → Work OS 확장 · Claude API       | ★★★★☆ |
| 16:55 | 휴식                                                |       |
| 17:05 | LV7② 검증 · 전달 · 다음 단계                              | ★★★★☆ |

# PDF 교안과 필수 실습자료

## PDF 교안

탭·선택 내용을 별도 페이지로  
클릭 없이 모두 확인  
프롬프트는 텍스트 선택·복사

## 필수자료 ZIP

grill-me 스킬 파일  
가상 CSV와 열 설명  
사용 안내·라이선스

## 내 프로젝트

빈 폴더에서 직접 제작  
탐색기에서 생성 파일 확인

# 실습 환경 점검표

☐

Gemini 로그인

gemini.google.com

☐

VS Code 실행

시작 메뉴 / 응용 프로그램

☐

Claude Code 패널에서 "안녕" → 답이 옴

왼쪽 \* 아이콘

☐

VS Code에서 새 프로젝트 폴더 열기

오늘의 작업 폴더

☐

크롬에서 로컬 웹앱 실행 준비

실행 브라우저 확인

☐

패널에서 Node 또는 지정 Python 버전 확인

LV2부터 사용

## Claude Code 응답 확인

Claude Code에 "안녕"을 보내세요.  
답변이 오면 대화를 시작할 수 있습니다.  
답변이 오지 않으면 강사에게 알려 주세요.

## 오늘의 약속

- 진행이 어려우면 강사에게 질문하기
- 질문하기 전에 현재 작업 저장하기
- 회사 자료·수치는 넣지 않기: 오늘 데이터는 전부 가상

# 개념 + 트렌드

## AI의 작동 원리와 오류

모델은 자동완성

책상 크기(컨텍스트)

대화 → 생각 → 일하는 AI

말뜻은 쓰기 직전에 다시

# 토큰: AI가 읽고 쓰는 단위

AI는 문장을 작은 조각인 토큰으로 나눠 처리합니다.

## 글자 수와 다른 단위

단어 하나가 한 토큰이거나  
여러 토큰으로 나뉠 수 있습니다.  
언어·모델에 따라 달라집니다.

## 입력 토큰 · 읽는 내용

내 프롬프트와 이전 대화,  
읽은 파일과 도구 결과도  
입력에 포함됩니다.

## 출력 토큰 · 만드는 내용

AI가 작성하는 답변과 코드.  
길게 생성할수록  
출력 토큰도 늘어납니다.

## 왜 알아야 할까요?

한 번에 다룰 수 있는 정보량과 사용량을 이해하는 기준입니다.  
API 비용은 모델과 입력·출력 토큰 사용량 등에 따라 달라집니다.

# 토큰 예측의 두 가지 성질

앞의 맥락을 바탕으로, 다음에 올 토큰을 예측합니다.

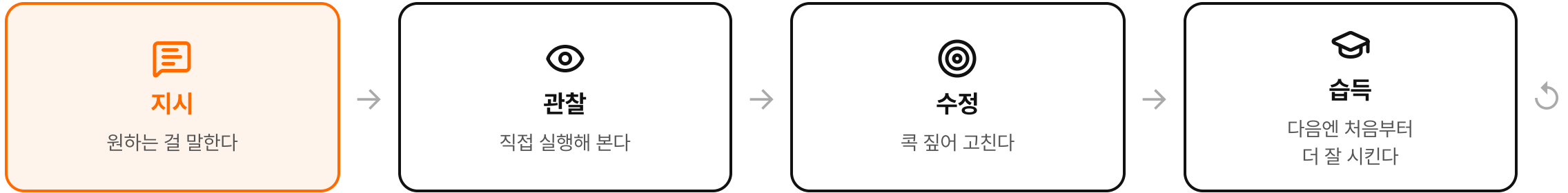
## ① 맥락에 따라 달라지는 답

“게임을 만들어줘”보다  
“폰에서 한 손으로 할 게임”의  
목표가 더 분명합니다.

## ② 자연스러워도 틀릴 수 있는 답

그렇듯한 다음 말을 예측해도  
사실이나 계산이 맞는다는  
보장은 없습니다.

# 사람의 역할: 결과 관찰과 수정



## 반복 피드백

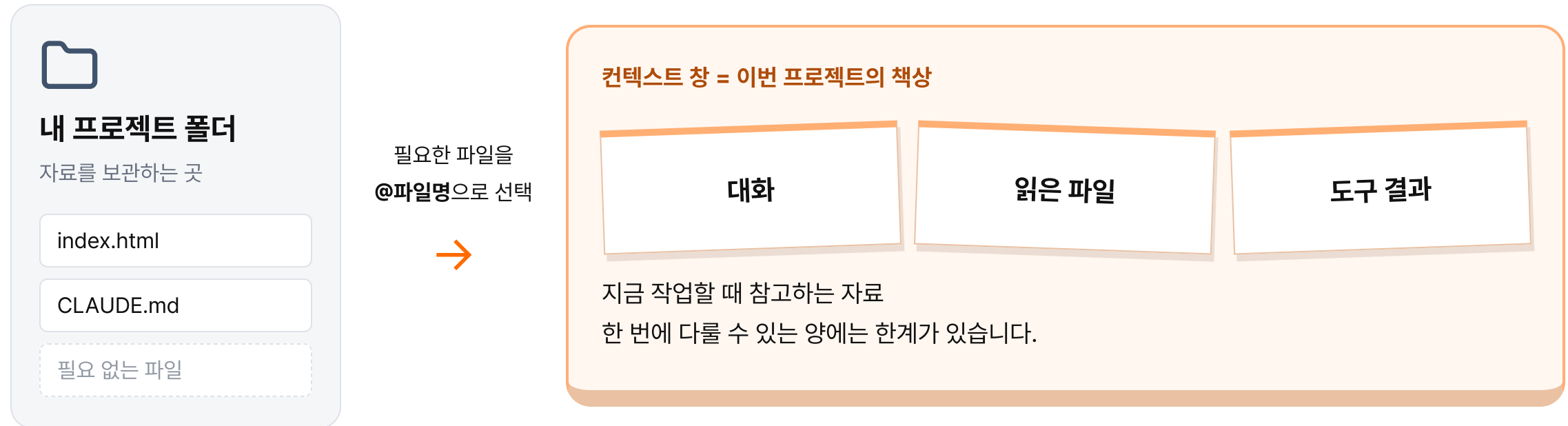
현재 모습과 원하는 변화를 알려 줍니다. **바뀐 결과를 실행해 확인**하고 다음 수정 프롬프트를 보냅니다.

## 사람에게 남는 일 두 가지

**무엇을 만들지 정하기(기획) · 나온 것을 확인하기(검증).** 코드를 쓰는 일은 AI가.

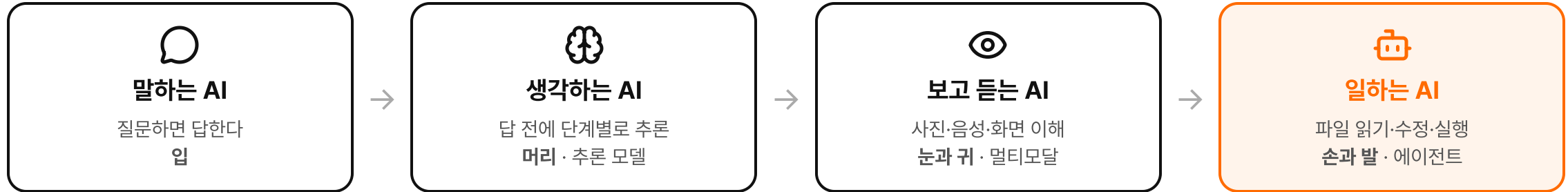
# 컨텍스트: 프로젝트의 작업 책상

구조도 · 자료 보관함과 지금 읽은 자료는 다릅니다



새 대화에는 목적·완료 기준을 다시 전달합니다. 긴 대화의 요약에서 세부 조건이 빠질 수 있습니다.  
폴더 전체가 자동으로 읽히지는 않습니다. 회사 자료는 올리지 않습니다.

# 모델의 진화: 언어·추론·시각·작업



## 에이전트는 이렇게 일합니다

목표 → 계획 → 파일 수정·실행 → 결과 확인. LV2부터 이 순서로 개발합니다.

# 실습의 두 가지 도구

## Gemini 웹 · Canvas

LV1: 게임 생성·실행

### 웹 브라우저에서 작업

한 줄 프롬프트 → 실행·HTML 보관

## VS Code + Claude Code

LV2-7: 게임 수정·웹앱 기획·제작·검증

### 프로젝트 폴더에서 작업

문서·피드백 → 구현 → 검증·수정

게임 수정부터 VS Code + Claude Code를 사용합니다.

# LEVEL 1

## 한 줄 게임

이번 레벨에서 만드는 것  
두 디자인으로 만들고 하나 골라 움직여 보기

도구  
Gemini 웹

난이도  
★☆☆☆☆

말뜻 알아보기

강사 한 장면

내 소재로 만들기

눌러서 확인

# Gemini의 최근 소식과 오늘 실습

① Argon 발표 · 이용 범위

② 오늘 실습 · Flash·Canvas

구글은 2026년 9월 30일 Gemini 4 Argon(아르곤)을 발표했습니다.

## 어떤 모델인가요?

구글은 복잡한 코딩과 업무 자료 분석,  
긴 작업을 이어가는 능력을  
강점으로 소개했습니다.

## 지금 웹에서 쓸 수 있나요?

현재는 제한된 테스터에 우선 제공됩니다.  
**일반 Gemini 웹에서는 아직  
사용할 수 없습니다.**

오늘 게임 실습은 Gemini 3.8 Flash로 진행합니다.

[Google 공식 발표 · 2026.09.30](#) · 이용 범위 확인: 2026.10.08

# Gemini의 최근 소식과 오늘 실습

① Argon 발표 · 이용 범위

② 오늘 실습 · Flash·Canvas

Flash는 속도와 효율을 중시하는 모델 계열입니다.

## 다양한 입력 이해

글·사진·음성·영상 이해.  
화면 예시를 주고  
원하는 모습을 설명합니다.

## 추론과 코딩

요구사항을 코드로 만들고  
복잡한 개발 작업을 수행.  
작은 게임부터 만들어 봅니다.

## 긴 자료와 단계 작업

많은 자료를 참고하고  
도구를 이용해 작업을 이어감.  
조건을 보태며 개선합니다.

## Canvas란?

Gemini 안에서 웹앱 코드를 만들고 바로 미리보기·실행하는 작업 공간입니다.  
이번 실습: **HTML 기반 게임 만들기 → 실행 → 파일 보관**

# AI가 정하는 게임의 모습

내가 말하지 않은 부분은 AI가 알아서 정합니다.

**“테트리스 만들어줘”라고 프롬프트로 보내면**

배경색·조작키·블록 속도는 아직 정하지 않았습니다.

**배경색**

예: 어두운 배경

**조작키**

예: 방향키로 블록 이동

**블록 속도**

예: 천천히 떨어지는 블록

AI가 정할 수 있는 예입니다. 실제로 만든 게임에서 확인합니다.

# HTML: 화면의 내용과 구조

## 화면에 무엇을 둘까

게임 제목 · 시작 버튼  
조작 안내 · 점수

### index.html

웹앱의 시작 파일

## 파일 하나여도 웹앱입니다

HTML 안의 **<style>**에 CSS,  
**<script>**에 JavaScript를  
함께 담을 수 있습니다.

이 경우 **HTML 파일 하나**를 열면  
디자인과 동작도 함께 적용됩니다.

수업의 단순 웹앱은 HTML 하나로도 충분합니다.

# CSS: 화면의 모양과 배치

## 무엇이 달라질까

글씨 크기 · 색 · 여백

버튼 크기 · 정렬 · 폰 화면 배치

## 게임의 예

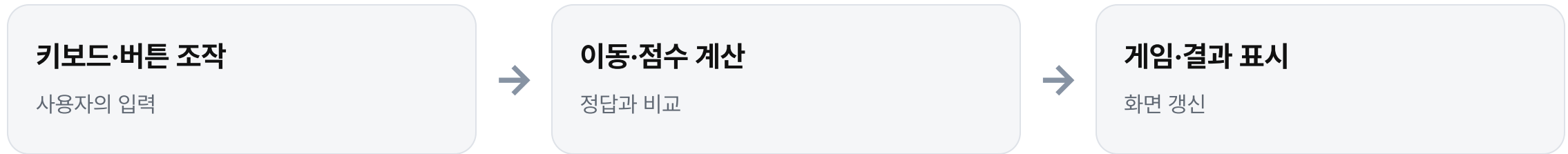
시작 버튼을 크게

점수는 굵게

휴대폰에서도 가로 넘침 없이

CSS는 같은 내용을 어떻게 보이게 할지 정합니다.

# JavaScript: 입력에 따른 동작



## 버튼이 보이는 것과 동작하는 것은 다릅니다

키보드나 버튼을 눌러 실제로 움직이는지,  
게임이 끝나면 점수가 맞는지 확인합니다.

**JavaScript는 눌렀을 때 무엇이 일어날지 정합니다.**

LV1



## 내 게임의 주제 선택



테트리스



블록 이동·회전·줄 제거를 확인



체스



말 선택·이동·차례를 확인



장기



말 이동·차례를 하나씩 확인



스네이크



먹이·몸 길이·벽 충돌을 정한다

자유 선택: 예시 밖의 게임·테마도 가능

[만들고 싶은 간단한 게임] 만들어줘

LV1



## 내 게임의 주제 선택



테트리스



블록 이동·회전·줄 제거를 확인



체스



말 선택·이동·차례를 확인



장기



말 이동·차례를 하나씩 확인



스네이크



먹이·몸 길이·벽 충돌을 정한다

자유 선택: 예시 밖의 게임·테마도 가능

테트리스 만들어줘

LV1



## 내 게임의 주제 선택



테트리스



블록 이동·회전·줄 제거를 확인



체스



말 선택·이동·차례를 확인



장기



말 이동·차례를 하나씩 확인



스네이크



먹이·몸 길이·벽 충돌을 정한다

자유 선택: 예시 밖의 게임·테마도 가능

체스 만들어줘

LV1



## 내 게임의 주제 선택



테트리스



블록 이동·회전·줄 제거를 확인



체스



말 선택·이동·차례를 확인



장기



말 이동·차례를 하나씩 확인



스네이크



먹이·몸 길이·벽 충돌을 정한다

자유 선택: 예시 밖의 게임·테마도 가능

장기 만들어줘

LV1



## 내 게임의 주제 선택



테트리스



블록 이동·회전·줄 제거를 확인



체스



말 선택·이동·차례를 확인



장기



말 이동·차례를 하나씩 확인



스네이크



먹이·몸 길이·벽 충돌을 정한다

자유 선택: 예시 밖의 게임·테마도 가능

스네이크 게임 만들어줘

# 같은 게임, 다른 디자인

새 채팅 두 개를 열고, 아래 프롬프트를 하나씩 넣어 비교해 보세요.

새 채팅 A · 포근한 파스텔

테트리스 웹게임을 만들어줘.  
크림색 배경에 코랄·민트 블록,  
둥근 카드와 여유 있는 간격으로.  
상단에 점수와 다음 블록을 보여줘.  
전체 분위기는 포근한 미니 게임처럼.

새 채팅 B · 네온 아케이드

테트리스 웹게임을 만들어줘.  
짙은 남색 배경에 청록·보라 네온 블록,  
은은한 빛 효과로 오락실 분위기를 내줘.  
점수와 다음 블록은 옆 패널에 보여줘.  
시작 버튼은 크고 눈에 띄게 해줘.

둘 다 실행해 보고 마음에 드는 하나를 고릅니다. 그 게임을 VS Code로 가져와 고칩니다.

# 강사 게임 화면의 관찰 기준

## 세 단계

- Gemini 입력창에서 **Canvas** 켜기
- 한 줄 붙여 넣고 보내기
- 미리보기에서 직접 해 보기

## Canvas 입력

## 강사 게임 결과

성훈님, 마이크를 사용하세요

테트리스 만들어줘

+  Canvas

Flash   

Canvas를 켜 입력창. 수강생은 자기 게임 이름으로 프롬프트를 보냅니다.

☐ 실행이 되나요?

☐ 규칙이 맞나요?

☐ 조작은 편한가요?

☐ 점수나 승패가 나오나요?

☐ 어색한 점 3개 를 메모했나요?

☐ 옆 사람에게 소개: 내가 고른 게임·바꾸고 싶은 점은?

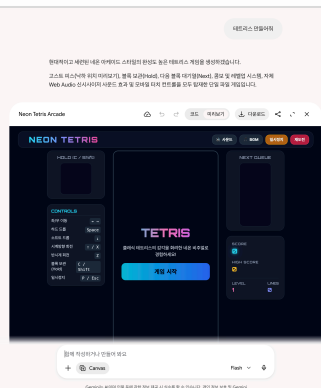
# 강사 게임 화면의 관찰 기준

## 세 단계

- Gemini 입력창에서 **Canvas** 켜기
- 한 줄 붙여 넣고 보내기
- 미리보기에서 직접 해 보기

Canvas 입력

강사 게임 결과



강사 시연 결과: 한 줄로 만든 테트리스 첫 화면. 클릭하면 확대됩니다.

☐ 실행이 되나요?

☐ 규칙이 맞나요?

☐ 조작은 편한가요?

☐ 점수나 승패가 나오나요?

☐ 어색한 점 3개 를 메모했나요?

☐ 옆 사람에게 소개: 내가 고른 게임·바꾸고 싶은 점은?

# 고른 게임의 **실행 확인**과 보관

Gemini · Canvas

## 게임 하나 선택

두 채팅의 결과에서



## 직접 조작

원하는 모습과 비교



## HTML 보관

LV2에서 이어서 수정

## 직접 해 볼 것

- 게임을 시작하고 이동·클릭·키보드로 조작합니다.
- 점수·승패가 예상대로 보이는지 확인합니다.
- 불편한 점이나 추가하고 싶은 기능 하나를 고릅니다.

# LEVEL 2

## 내 게임의 수정과 기능 추가

이번 레벨에서 만드는 것

내가 만든 게임을 프로젝트에 옮겨 변화 하나 구현·확인

도구

VS Code + Claude  
Code

난이도



말뜻 알아보기

강사 한 장면

내 소재로 만들기

눌러서 확인

# Claude의 Chat · Cowork · Code

같은 Claude를 어떤 일을 맡길지에 따라 다른 방식으로 사용합니다.

## Chat · 대화하며 생각 정리

질문 · 자료 읽기 · 글 초안

예: "이 아이디어의 약점은?"

답을 읽고 다음 질문으로 이어갑니다.

## Cowork · 파일과 업무 처리

선택한 폴더와 도구로 작업

예: "이 자료를 보고서로 정리해줘"

실행한 결과물을 검토합니다.

## Code · 프로젝트 개발

코드 파일 읽기 · 수정 · 실행

예: "게임에 점수판을 추가하자"

오늘은 VS Code에서 사용합니다.

## 오늘부터는 Claude Code

말로 기획하고, 프로젝트의 파일을 수정하며, 실행 화면을 직접 확인합니다.

세 방식의 기능은 겹칠 수 있으며 접근할 파일·도구의 범위는 설정에 따라 다릅니다.

출처: [Claude](#) · [Cowork](#) · [Claude Code](#)

# Claude 모델과 Claude Code의 역할

Claude는 AI 모델 · Claude Code는 파일을 읽고 수정·실행하는 개발 도구

## Haiku · 빠른 처리

짧은 분류·정보 추출처럼  
많은 작은 작업을  
빠르게 처리할 때.

## Sonnet · 속도와 성능

범위가 정해진 기능 제작,  
문서 작성과 오류 수정.  
실습: **Sonnet 5.5**

## Opus · 복잡한 작업

여러 단계의 어려운 문제,  
오래 이어지는 개발과  
설계·분석 작업에 활용.

## Claude Code에서 하는 일

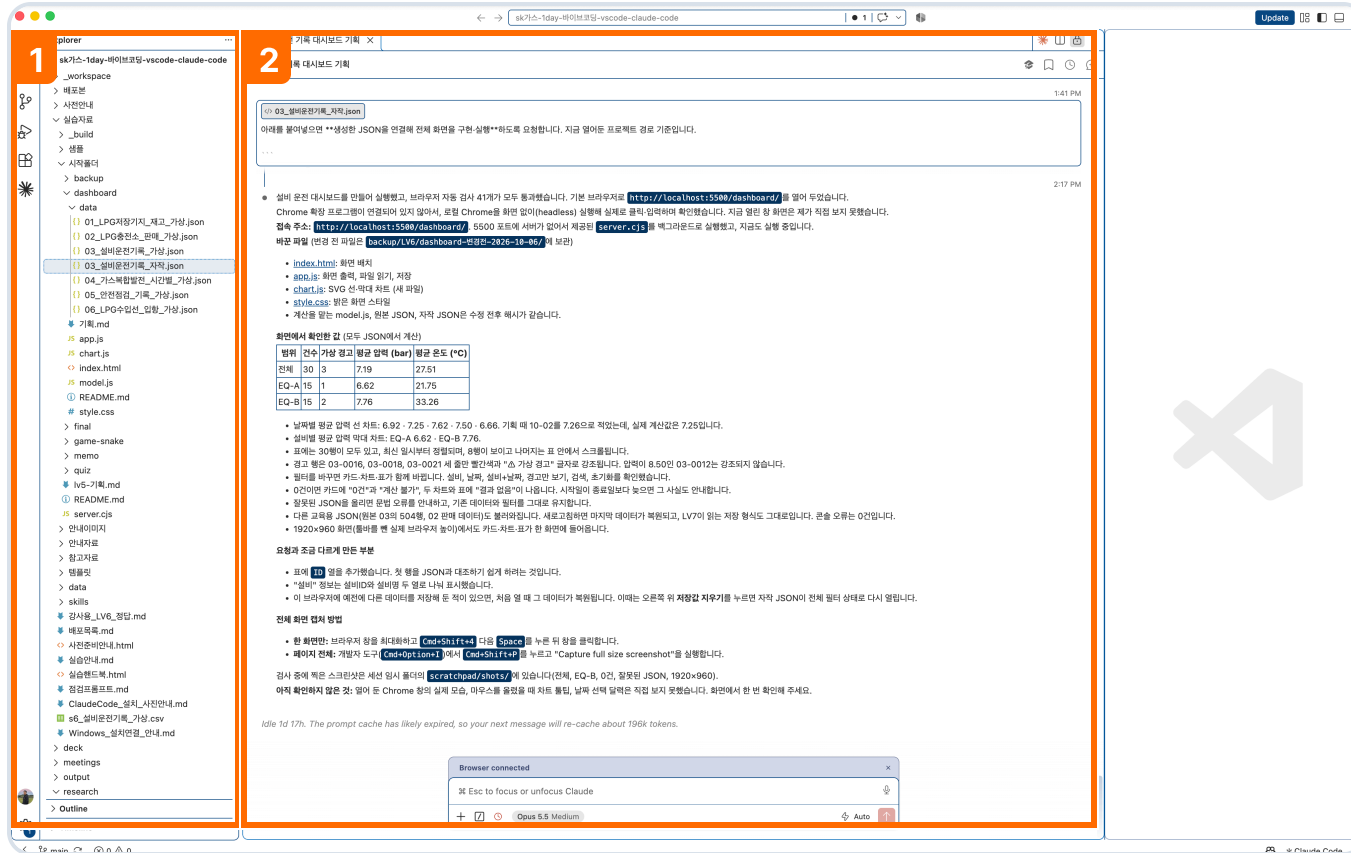
대화로 목표를 전달하면 프로젝트의 **파일 읽기 → 수정 → 실행·검증**을 수행합니다.  
웹앱은 localhost에서 열고, 학습자도 직접 눌러 확인합니다.

## VS Code: 화면 구역과 기본 조작

## 전체 위치

## ① 탐색기

## ② Claude Code



## ① 탐색기

## 폴더를 펼치고 파일 선택

## ② Claude Code 패널

## AI의 답변과 내 프롬프트

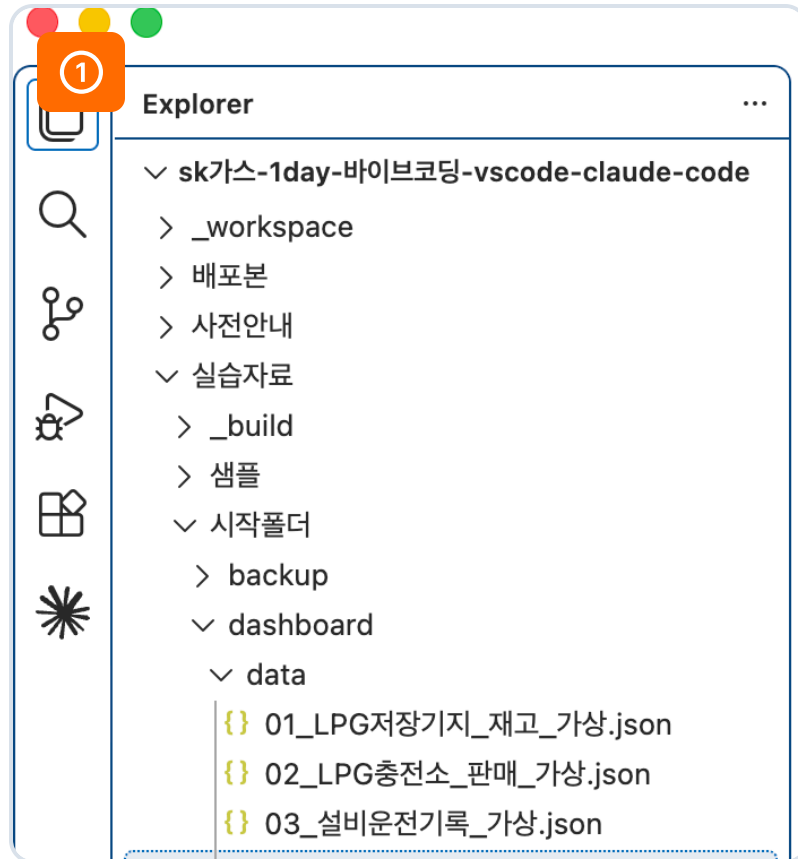
사용자 실제 캡처·파일 편집기는 이 사진에서 열려 있지 않습니다.

# VS Code: 화면 구역과 기본 조작

전체 위치

① 탐색기

② Claude Code



## 폴더·파일 선택

왼쪽 파일 아이콘 클릭

Windows: Ctrl+Shift+E

Mac: ⌘+Shift+E

폴더 앞 화살표로 펼치기

파일 이름을 눌러 내용 열기

사진의 경로는 강사 프로젝트입니다.

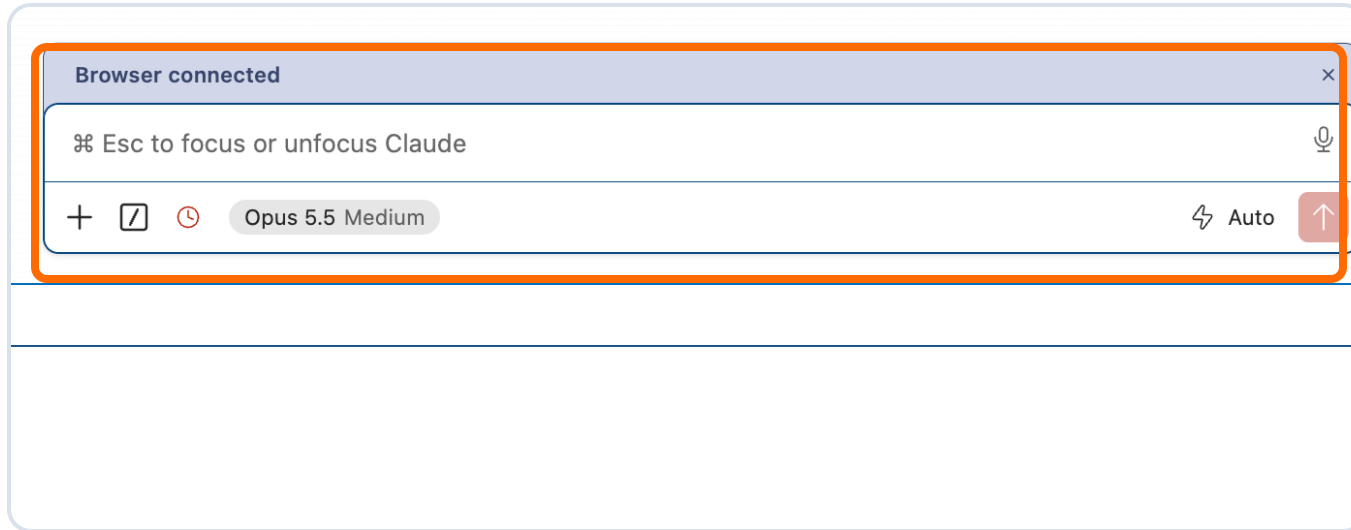
실습에서는 직접 만든 프로젝트 폴더를 엽니다.

# VS Code: 화면 구역과 기본 조작

전체 위치

① 탐색기

② Claude Code



## ② 패널 아래 입력창

왼쪽 Claude Code 아이콘 클릭

입력창에 프롬프트 작성

화살표 버튼으로 보내기

AI 답변은 위쪽에 표시됩니다.

파일 변경은 탐색기에서 확인합니다.

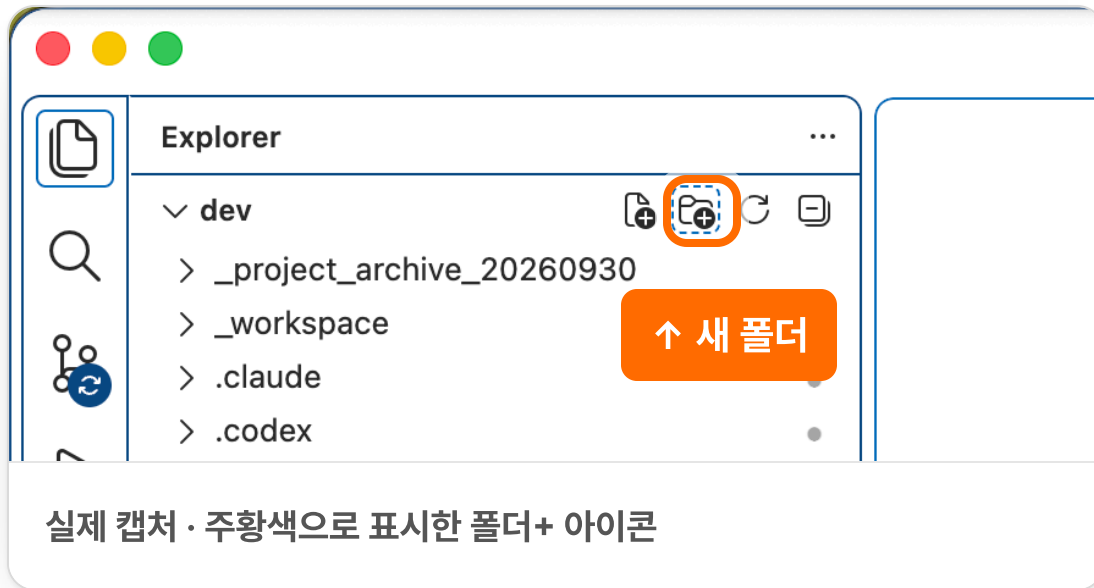
사진의 모델·모드 표시는 촬영 당시 상태입니다. 수업 설정은 이어지는 시작 장에서 확인합니다.

# 새 프로젝트의 **작업 폴더** 생성

① 새 폴더 만들기

② 새 창으로 드래그

③ 작업 위치 확인



## 새 폴더 생성

1. 표시한 **새 폴더 아이콘** 클릭
2. **vibe-lab** 입력
3. **Enter**로 생성

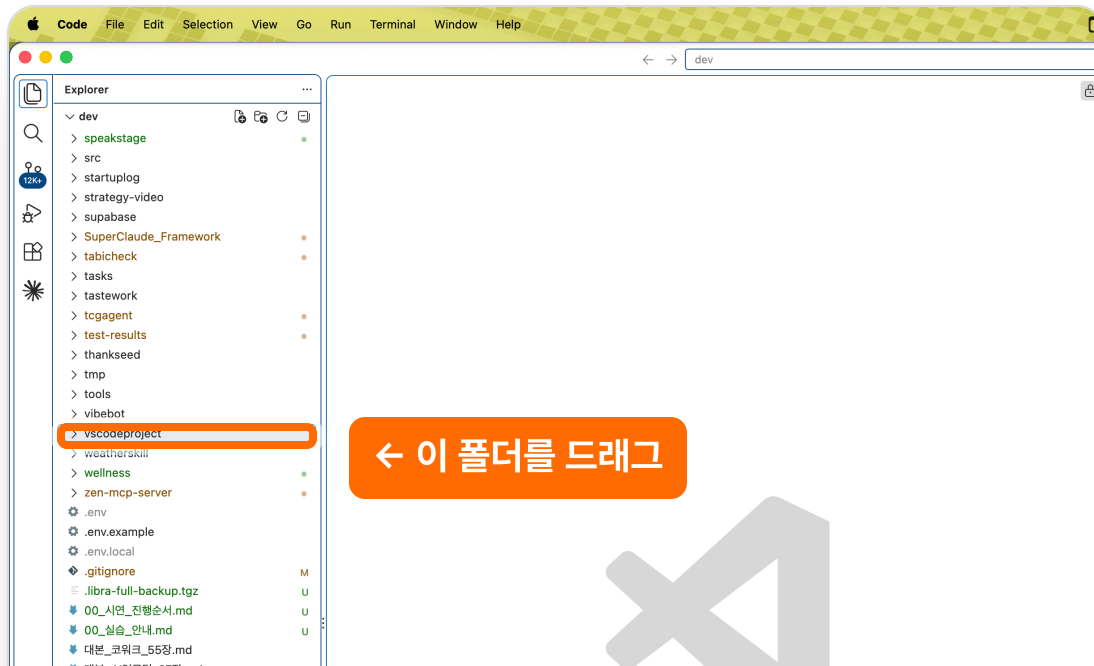
현재 열린 폴더 아래에  
새 폴더가 만들어집니다.

# 새 프로젝트의 작업 폴더 생성

① 새 폴더 만들기

② 새 창으로 드래그

③ 작업 위치 확인



캡처의 vscodeproject = 실습의 vibe-lab 예시

## 새 창에서 프로젝트 시작

1. **File** → **New Window**로 새 창 열기
2. 만든 폴더를 새 창으로 드래그
3. 가운데 빈 편집 영역에 놓기

새 창: Windows Ctrl + Shift + N

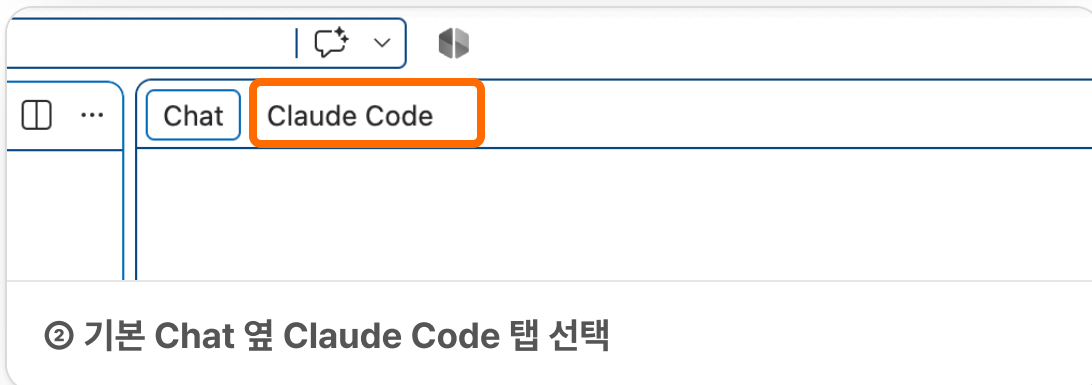
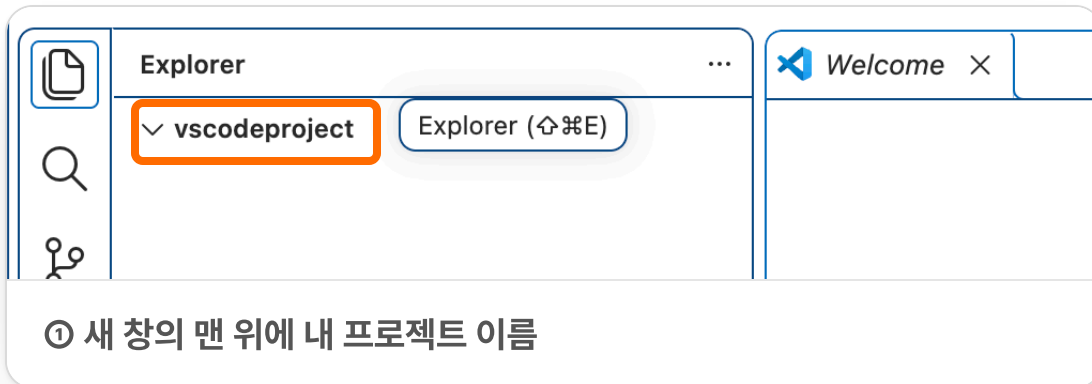
Mac ⌘ + Shift + N

# 새 프로젝트의 작업 폴더 생성

① 새 폴더 만들기

② 새 창으로 드래그

③ 작업 위치 확인



## 이 폴더에서 Claude Code 사용

1. 맨 위 이름이 **내 프로젝트**인지 확인
2. 오른쪽 **Claude Code** 탭 클릭
3. 새 대화에서 **게임 파일 가져오기**

실습 이름: vibe-lab

캡처 예시 이름: vscodeproject

# Claude Code의 **작업 승인 모드**

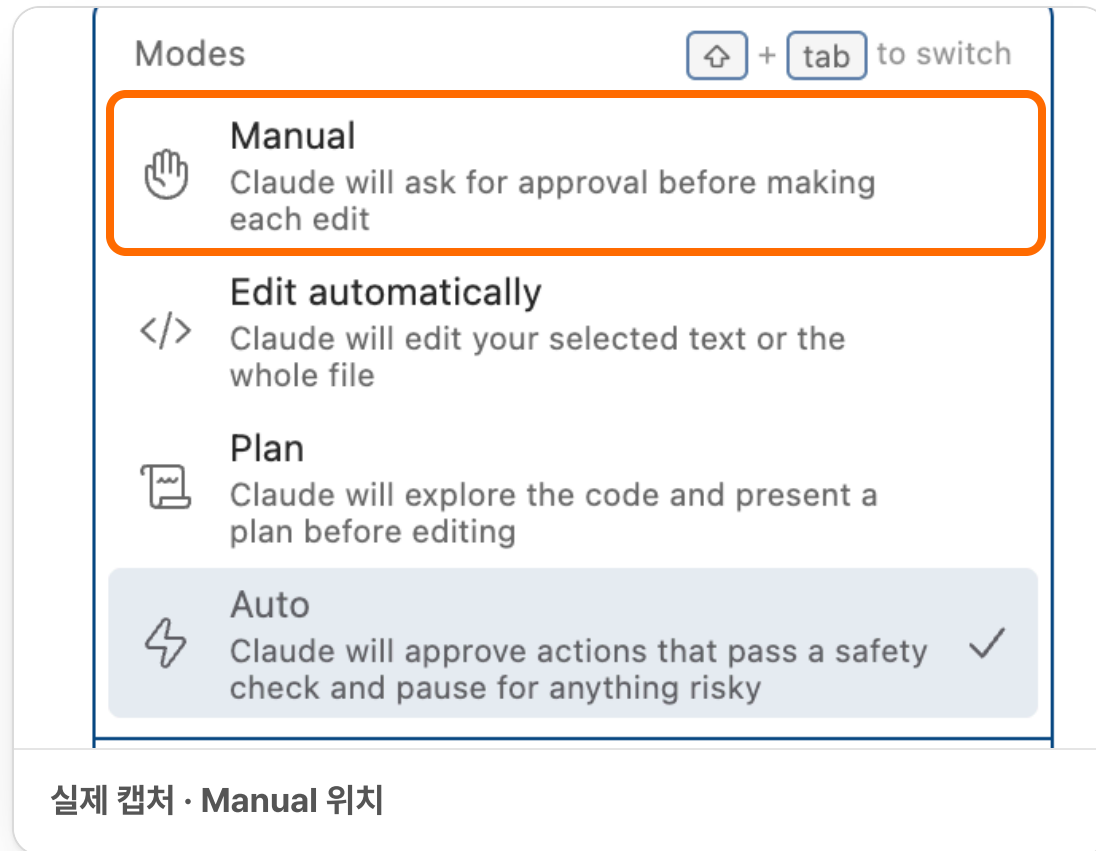
입력창 오른쪽 아래의 모드 이름을 눌러 선택합니다.

① **Manual**

② Edit automatically

③ Plan

④ Auto



## 파일을 바꾸기 전에 직접 승인

수정할 내용과 실행할 명령을 확인하고  
허용 여부를 내가 결정합니다.

### 사용할 때

처음 개발하거나 변경 내용을 하나씩 보고 싶을 때.

# Claude Code의 **작업 승인 모드**

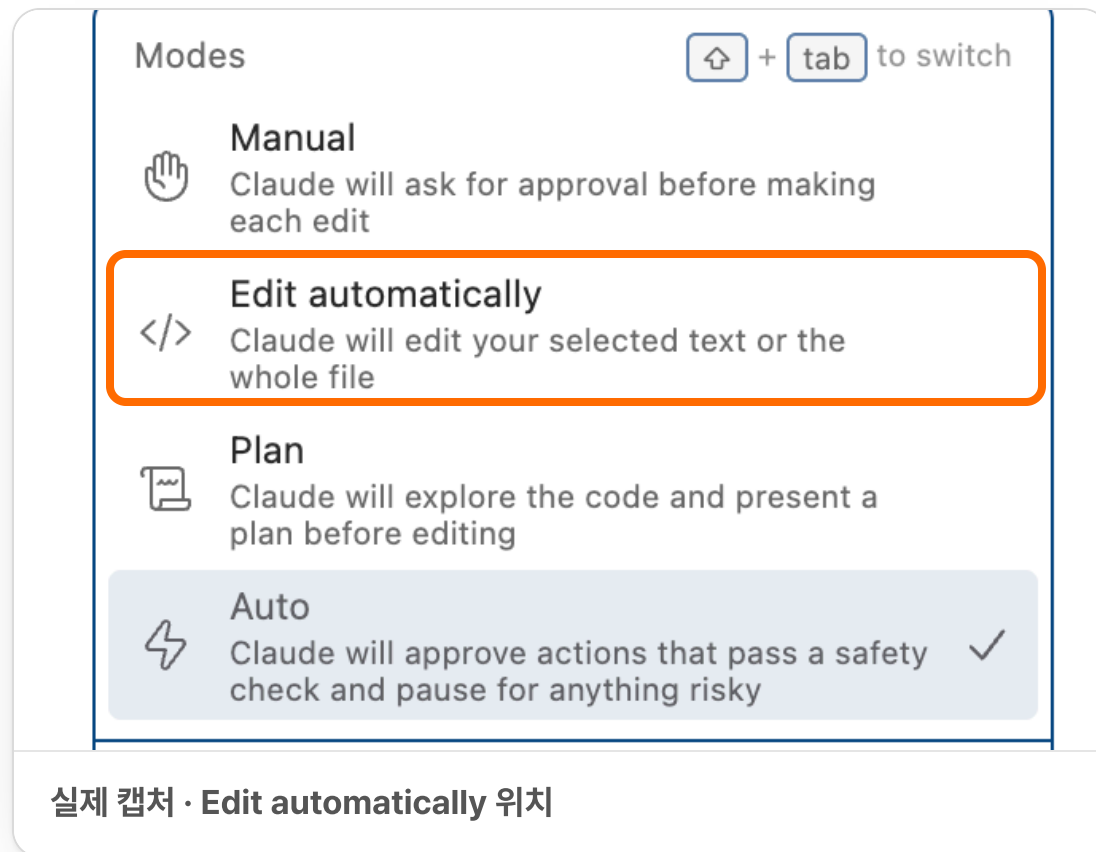
입력창 오른쪽 아래의 모드 이름을 눌러 선택합니다.

① Manual

② **Edit automatically**

③ Plan

④ Auto



## 파일 수정 자동 허용

파일 수정을 매번 승인하지 않고  
Claude Code가 바로 반영합니다.

### 사용할 때

변경 범위를 정한 뒤 여러 수정을 진행할 때.

# Claude Code의 **작업 승인 모드**

입력창 오른쪽 아래의 모드 이름을 눌러 선택합니다.

① Manual

② Edit automatically

③ **Plan**

④ Auto

Modes

 +  to switch



**Manual**

Claude will ask for approval before making each edit



**Edit automatically**

Claude will edit your selected text or the whole file



**Plan**

Claude will explore the code and present a plan before editing



**Auto**

Claude will approve actions that pass a safety check and pause for anything risky



실제 캡처 · Plan 위치

## 구현 전에 계획 검토

필요한 내용을 조사하고 계획을 제안합니다.

계획을 읽고 피드백한 뒤 구현으로 넘어갑니다.

## 사용할 때

아이디어 인터뷰·화면 설계·구현 계획을 정할 때.

# Claude Code의 **작업 승인 모드**

입력창 오른쪽 아래의 모드 이름을 눌러 선택합니다.

① Manual

② Edit automatically

③ Plan

④ **Auto**

Modes

 + **tab** to switch



**Manual**

Claude will ask for approval before making each edit



**Edit automatically**

Claude will edit your selected text or the whole file



**Plan**

Claude will explore the code and present a plan before editing



**Auto**

Claude will approve actions that pass a safety check and pause for anything risky



실제 캡처 · Auto 위치

## 작업의 위험을 자동 판단

검사를 통과한 작업은 진행합니다.

위험한 작업은 멈추거나 승인을 묻습니다.

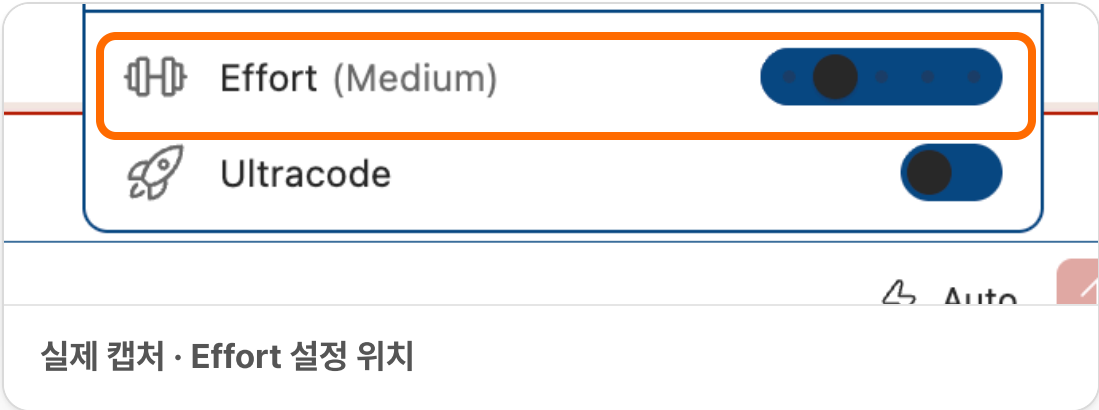
### 사용할 때

자동으로 진행하되 위험한 작업을 확인하고 싶을 때.

# Claude Code의 Effort와 Ultracode

Effort는 추론 노력 수준, Ultracode는 작업 흐름을 구성하는 설정입니다.

- ① Effort · 추론 노력
- ② Ultracode · 작업 흐름



**얼마나 깊게 검토할지 조절**

작업이 어려울수록 수준을 높입니다.  
높이면 시간과 사용량이 늘 수 있습니다.

| Effort | 사용 예          |
|--------|---------------|
| low    | 글씨·색·이름 수정    |
| medium | 범위가 정해진 기능 구현 |
| high   | 복잡한 기획·오류 분석  |

**수업 모델: Sonnet**

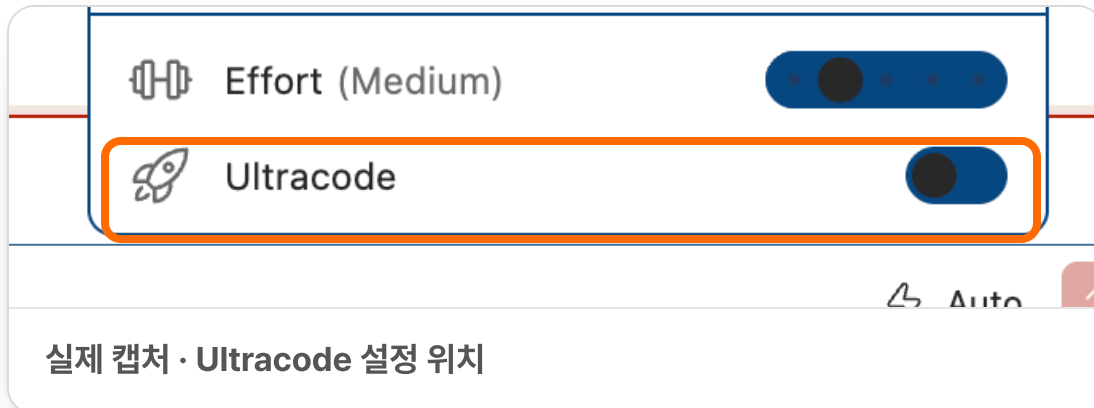
**기본 medium에서 필요할 때 조절**

# Claude Code의 Effort와 Ultracode

Effort는 추론 노력 수준, Ultracode는 작업 흐름을 구성하는 설정입니다.

① Effort · 추론 노력

② Ultracode · 작업 흐름



## 복잡한 작업의 흐름 구성

필요한 단계와 역할을 구성하고  
여러 에이전트를 활용해 진행합니다.

## 필요한 작업에서 켜기

여러 파일 수정·복잡한 오류 분석·  
계획과 검증이 함께 필요한 작업

Effort에서 정한 수준으로 작업합니다.  
시간과 사용량이 늘 수 있습니다.

수업의 간단한 수정은 꺼 뒀도 됩니다.  
작업 승인 방식은 모드에서 따로 정합니다.

# Rewind: 이전 시점의 복구

잘못 바꿨다면 문제가 생기기 전 시점으로 돌아갑니다.

① /rewind 입력

② 돌아갈 메시지 선택

Context

Rewind

/rewind|

🎤

+ 📄 ⌚ Opus 5.5 Medium ⚡ Auto ↩

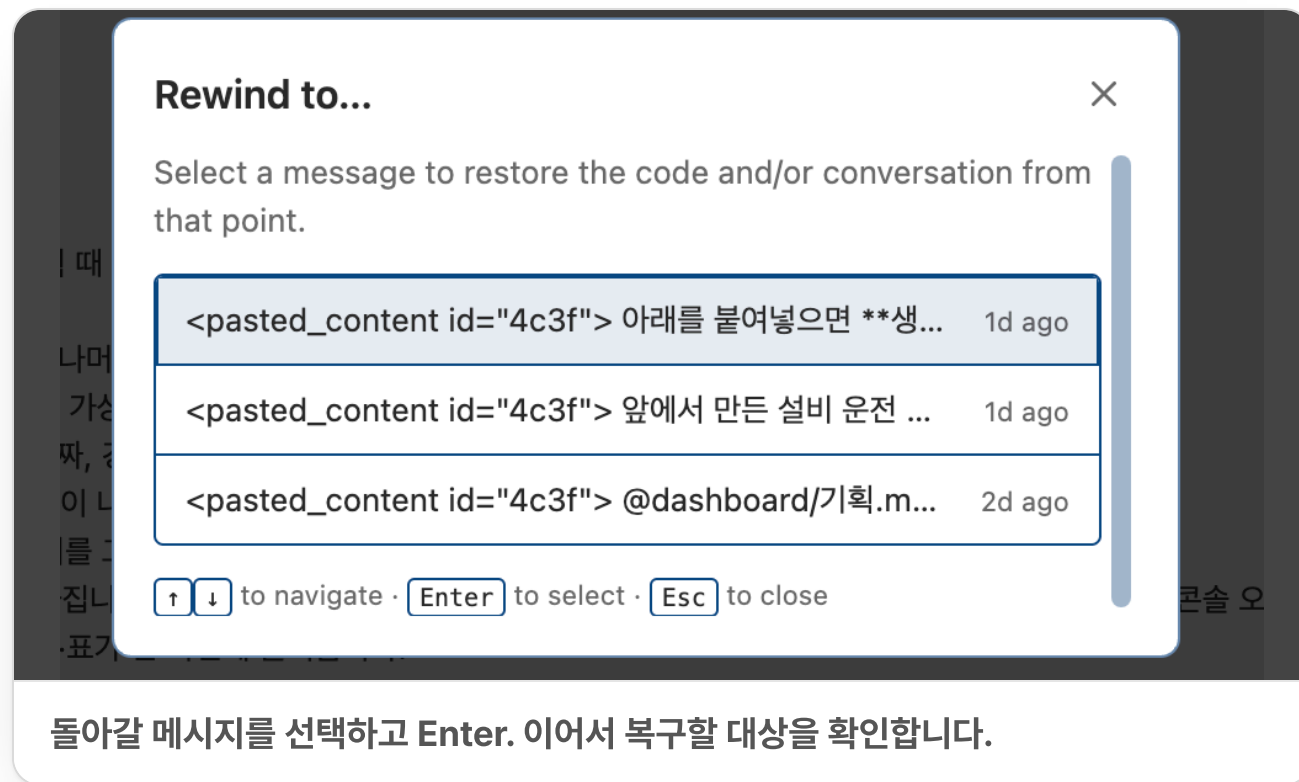
입력창에 /rewind를 쓰고 위의 Rewind를 선택합니다.

# Rewind: 이전 시점의 복구

잘못 바꿨다면 문제가 생기기 전 시점으로 돌아갑니다.

① /rewind 입력

② 돌아갈 메시지 선택

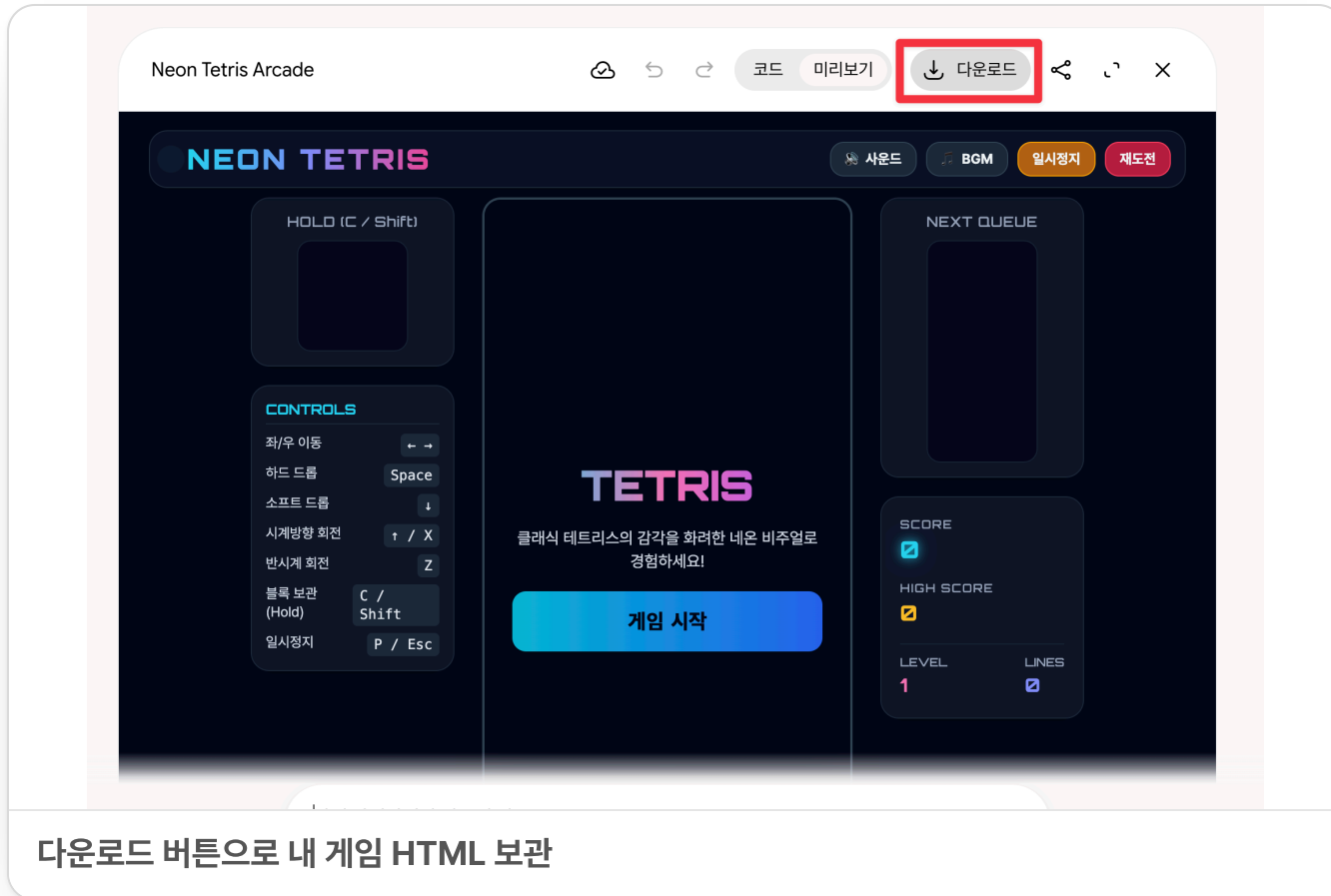


# Gemini 게임의 프로젝트 가져오기

① HTML 다운로드

② 폴더에 넣기

대안 · 코드 복사



다운로드 버튼으로 내 게임 HTML 보관

## 내 게임 파일 받기

1. Gemini의 다운로드 클릭
2. 다운로드한 HTML 파일 찾기
3. 앞에서 만든 프로젝트 폴더로 옮기기

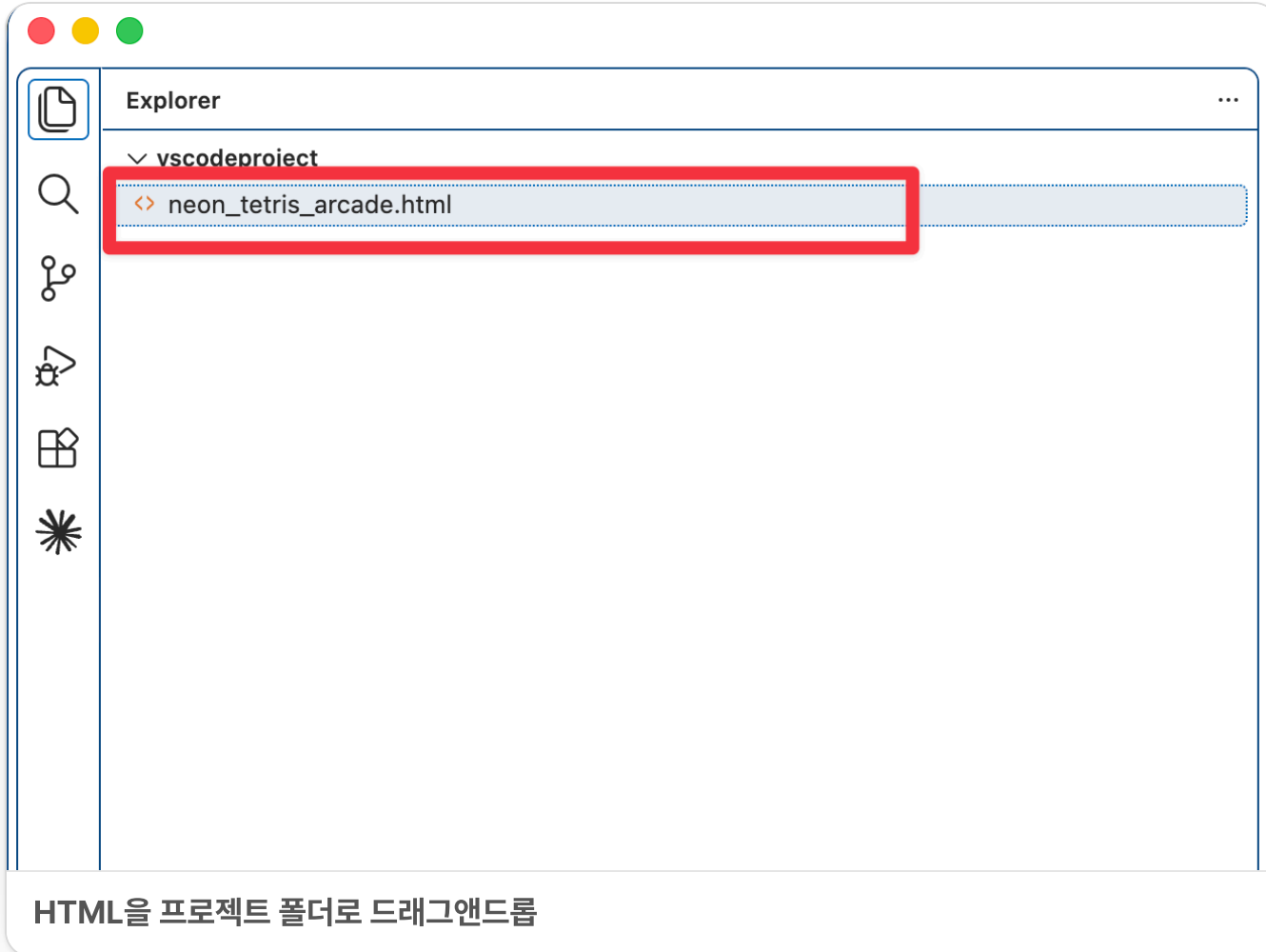
파일 이름은 내가 만든 게임에 따라 다릅니다.

# Gemini 게임의 프로젝트 가져오기

① HTML 다운로드

② 폴더에 넣기

대안 · 코드 복사



## 파일이 들어왔는지 확인

1. 다운로드한 HTML을 프로젝트 폴더에 넣기
2. VS Code 탐색기에 파일 이름이 보이는지 확인

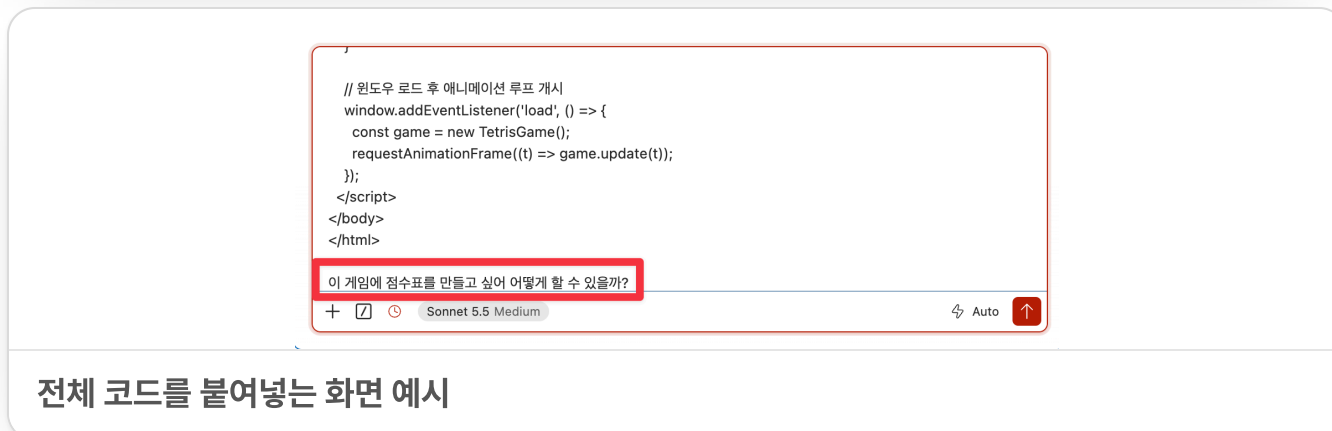
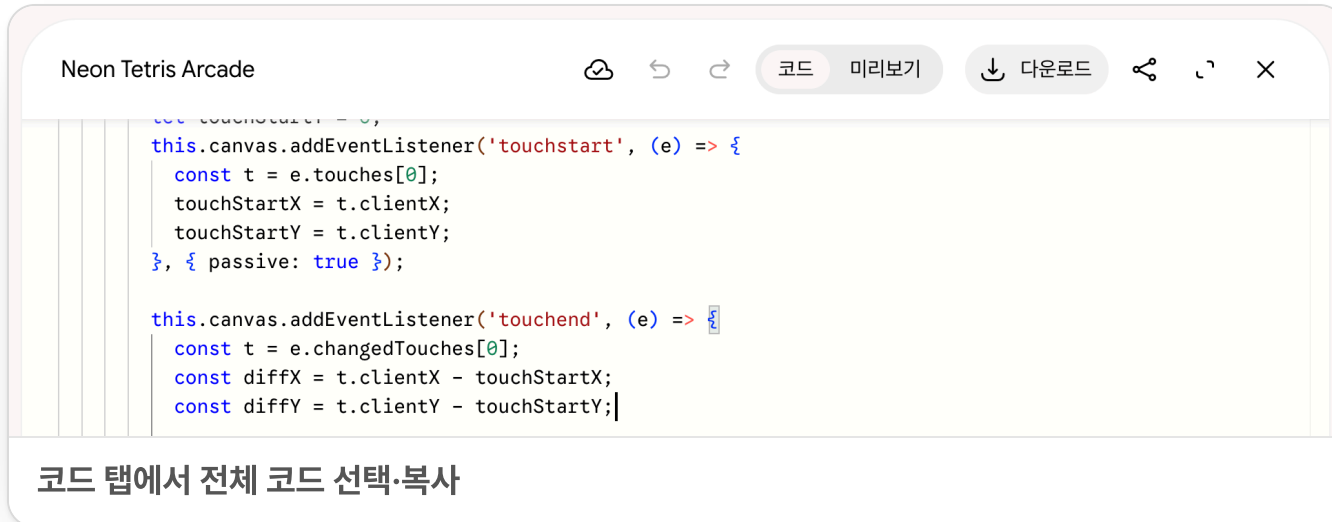
다음 장에서 이 파일을 AI에게 지정합니다.

# Gemini 게임의 프로젝트 가져오기

① HTML 다운로드

② 폴더에 넣기

대안 · 코드 복사



## 다운로드 대신 코드 복사

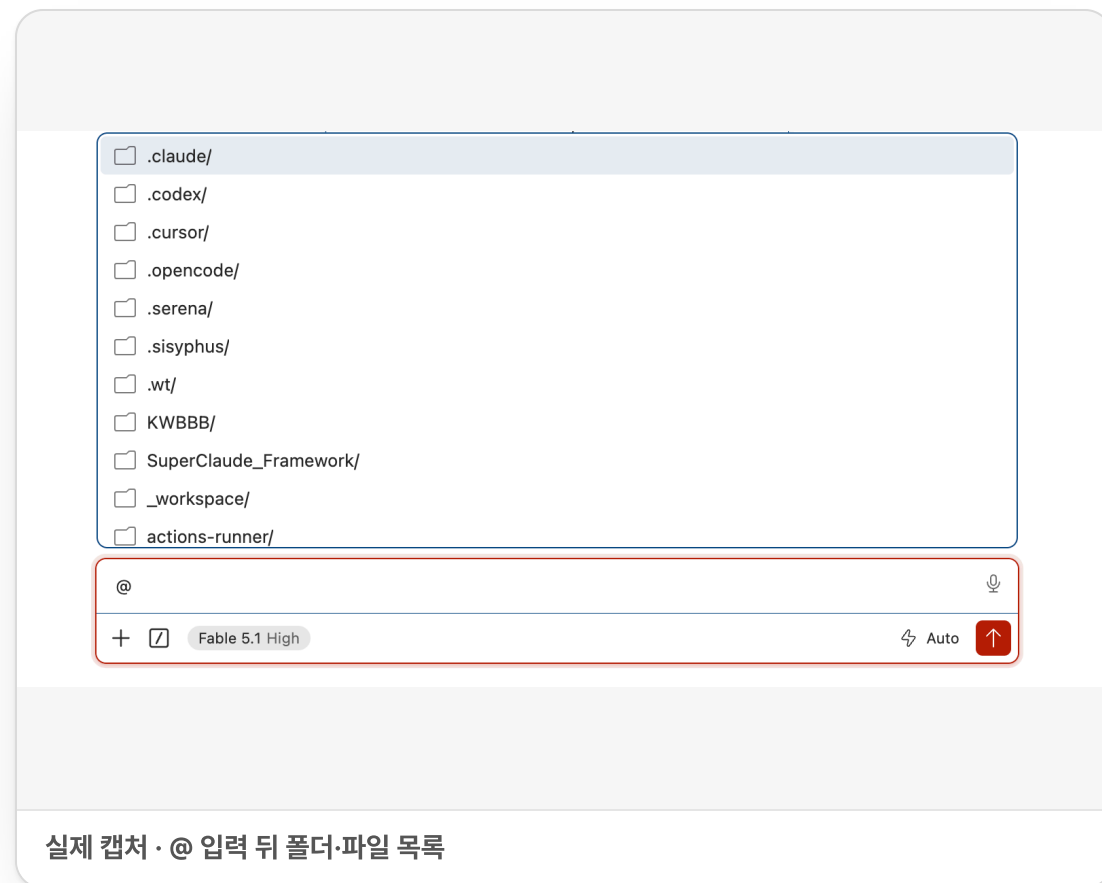
Gemini의 코드 탭에서  
전체 HTML 코드를 복사해  
Claude Code 입력창에 붙입니다.

붙인 코드 아래에 보낼 프롬프트

붙인 전체 코드를 프로젝트의 게임 HTML 파일로  
저장해줘.  
기존 게임 기능은 유지하고, 저장한 파일 이름을  
알려줘.

# 골뱅이(@)를 이용한 파일·폴더 지정

게임 파일을 가져온 뒤, AI가 봐야 할 대상을 골라 프롬프트를 보냅니다.



## ① @ 입력 → 대상 선택

입력창에 @ 입력

이름을 이어 입력해 찾기

목록에서 파일 또는 폴더 선택

가져온 게임 파일 지정

@neon\_tetris\_arcade.html 이 게임의 현재 기능을 짧게 설명해줘.

파일 하나: @neon\_tetris\_arcade.html

폴더: @vibe-lab/

# 컴포넌트: 직접 누르는 화면 요소

입력창 · 버튼 · 카드 · 상태 표시를 같은 화면에서 조작해 봅니다.

## 입력창 · 메모

예: 오늘 만든 퀴즈

## 드롭다운 · 상태

작성 중

카드에 반영

메모를 쓰고 버튼을 눌러 보세요.

## 카드 · 기록 하나

아직 입력한 기록이 없습니다.

작성 중

# 컴포넌트: 직접 누르는 화면 요소

입력창 · 버튼 · 카드 · 상태 표시를 같은 화면에서 조작해 봅니다.

## 입력창 · 메모

예: 오늘 만든 퀴즈

## 드롭다운 · 상태

작성 중

카드에 반영

메모를 입력해 주세요.

## 카드 · 기록 하나

아직 입력한 기록이 없습니다.

작성 중

# 컴포넌트: 직접 누르는 화면 요소

입력창 · 버튼 · 카드 · 상태 표시를 같은 화면에서 조작해 봅니다.

## 입력창 · 메모

오늘 만든 퀴즈

## 드롭다운 · 상태

완료

카드에 반영

카드에 반영했습니다. 이 화면의 시연용 기록입니다.

## 카드 · 기록 하나

오늘 만든 퀴즈

완료

# 내 게임의 변화 구체화와 구현

VS Code + Claude Code

변화 예시

① 수정 계획

② 구현

③ 실행 후 확인

## 지금의 불편

게임 화면

SCORE 120

게임 중 점수를 찾기 어렵습니다.

## 원하는 결과

화면 상단

점수 120

점수가 한눈에 보이면 좋겠습니다.

설명용 예시입니다. 수정 뒤에는 점수가 잘 보이는지, 게임 조작이 그대로인지 직접 확인합니다.

# 내 게임의 변화 구체화와 구현

VS Code + Claude Code

변화 예시

① 수정 계획

② 구현

③ 실행 후 확인

Plan · @로 내 게임 파일 지정 후

지금 게임은 [현재 모습]인데, [원하는 변화]로 바꾸고 싶어.  
더 필요한 정보가 있다면 내게 물어서  
수정 계획부터 제안해줘.  
내가 확인하면 그 계획대로 수정하고 실행하자.

## 내 목표에 맞게 바꾸기

화면 수정: "점수 글씨를 더 크게"

기능 추가: "게임이 끝나면 점수표에 기록"

# 내 게임의 변화 구체화와 구현

VS Code + Claude Code

변화 예시

① 수정 계획

② 구현

③ 실행 후 확인

질문에 답변

범위와 동작 결정



수정 계획 확인

내 의도와 비교



구현

구현 모드에서 진행

Manual 등 구현 모드 · 계획 승인 후

확인한 계획대로 게임을 수정해줘.

# 내 게임의 변화 구체화와 구현

VS Code + Claude Code

변화 예시

① 수정 계획

② 구현

③ 실행 후 확인

다음 장에서 로컬로 실행한 뒤 아래 항목을 확인합니다.

**“완료했습니다”라는 답 뒤에도 직접 눌러 확인합니다.**

## 원하는 변화

글씨가 잘 보이나요?  
추가한 점수표가 작동하나요?

## 기존 기능

이동·조작·점수가  
수정 전처럼 작동하나요?

## 관찰 결과로 피드백

기대한 결과와 실제 결과의  
차이를 알려 다시 수정합니다.

다크 모드를 바꿨다면 글자·입력창·버튼까지 확인합니다. 새로고침 후에도 적용되는지 봅니다.

# localhost: 내 PC의 웹앱 주소

만든 웹앱을 내 PC에서 실행하고, 브라우저로 열어 봅니다.

## localhost는 무엇인가요?

지금 사용하는 내 PC를 가리킵니다.

Claude Code가 로컬 서버를 실행하면

그 주소로 웹앱을 열 수 있습니다.

## 주소를 읽는 예

**http://localhost:5500**

localhost → 내 PC

5500 → 서버의 포트 번호

구현 후 Claude Code에 보내기

로컬에서 실행 가능하게 해줘.

브라우저로 열 수 있는 주소도 알려줘.

포트 번호는 예시입니다. AI가 알려준 실제 주소를 열고, 앞 장의 확인 항목을 직접 눌러 봅니다.

# LEVEL 3

## VS Code의 내 주제 퀴즈

이번 레벨에서 만드는 것

인터뷰 → IA → PRD → 퀴즈 구현 → 검증·수정

도구

VS Code + Claude  
Code

난이도



목표·인터뷰

내가 승인

구현

직접 확인·수정

# 만들기 전에 반박시켜 보기

좋다는 말만 듣고 만들기보다, 약한 가정을 먼저 살펴봅니다.

Plan · 내 아이디어로 짧게 연습

내 아이디어는 [내 주제] OX 퀴즈 웹앱이야.

약한 가정과 실패할 이유를 들어 반박해줘.

중요한 반론부터 하나씩 물어봐. 내 답을 듣고 다시 검토해줘.

**AI의 반론도 내가 판단합니다**

질문에 답하면서 아이디어를 고칩니다. 반론을 그대로 정답으로 받아들이지 않습니다.

# 퀴즈로 읽는 IA와 PRD

IA는 화면의 지도, PRD는 그 화면이 어떻게 작동할지 정한 조건입니다.

## 시작

주제 · 시작 버튼



## 문제

질문 · O/X



## 해설

답을 고른 뒤



## 결과

점수 · 다시 풀기

### ① 화면 IA를 먼저 확인

무엇을 보고 어디로 이동할까요?  
빠진 화면이나 버튼은 없나요?

### ② 조건 PRD를 인터뷰로 결정

해설은 언제 보일까요?  
틀린 문제는 어떻게 다시 풀까요?

설명용 네 칸입니다. 내 인터뷰 답에 맞게 고친 뒤 승인합니다.

# 내 주제 OX퀴즈의 기획·구현·검증 루프

## Claude Code 퀴즈 실습

① 인터뷰·IA

② PRD·피드백

③ 구현·확인

④ 수정·재확인

Plan · 나를 인터뷰하고 IA 작성

내 관심 주제로 OX퀴즈 웹앱을 만들고 싶어.

한 번에 질문 하나씩 나를 인터뷰해서 IA를 작성해줘.

IA를 내가 검토하고 승인하면,

추가 인터뷰로 PRD를 구체화하자.

목표를 말하고 AI의 질문에 답하며 기획을 구체화합니다(메타프롬프팅).

첫 문장은 내 목표로 바꾸고, IA에서 빠진 흐름을 피드백합니다.

# 내 주제 OX퀴즈의 기획·구현·검증 루프

## Claude Code 퀴즈 실습

① 인터뷰·IA

② PRD·피드백

③ 구현·확인

④ 수정·재확인

Plan · IA를 바탕으로 PRD 구체화

내가 승인한 IA를 바탕으로 PRD를 작성하고 싶어.

나를 인터뷰해서 동작과 완료 조건을 구체화해줘.

PRD를 내가 검토하고 수정할 수 있게 보여줘.

확인하면 구현 계획을 작성해줘.

PRD에서 내 의도와 다른 조건 하나를 찾아 수정해 봅니다.

예: “오답은 전체 문제가 아니라 틀린 문제만 다시 풀게 해줘.” 수정본을 확인한 뒤 승인합니다.

# 내 주제 OX퀴즈의 기획·구현·검증 루프

## Claude Code 퀴즈 실습

① 인터뷰·IA

② PRD·피드백

③ 구현·확인

④ 수정·재확인

승인 뒤 Manual · 구현과 실행

확인한 문서를 저장하고 이 기획대로 퀴즈 웹앱을 만들어줘.  
로컬에서 실행해줘.

O/X를 선택하고 해설·점수가 기획대로 보이는지 확인합니다.

# 내 주제 OX퀴즈의 기획·구현·검증 루프

## Claude Code 퀴즈 실습

① 인터뷰·IA

② PRD·피드백

③ 구현·확인

④ 수정·재확인

같은 대화 · 관찰한 결과로 피드백

지금은 답을 고르기 전에 해설이 보이는데,  
나는 O/X를 고른 뒤에만 보이게 바꾸고 싶어.  
더 필요한 정보는 내게 물어서 수정 계획부터 제안해줘.  
내가 확인하면 그 계획대로 수정하고 실행하자.

원하는 변화를 말하고 질문에 답합니다. 계획을 확인한 뒤 구현하고 직접 다시 눌러 봅니다.

# LEVEL 4

## 같은 퀴즈에 기능 하나

이번 레벨에서 만드는 것

추가할 기능 인터뷰 → 수정 계획 확인 → 구현·검증

도구

VS Code + Claude  
Code

난이도



목표·인터뷰

내가 승인

구현

직접 확인·수정

# 인터뷰 기반 기능·디자인 변경

① 기획 프롬프트·검토

② /plan 입력 사진

③ 명령 설명 사진

**Plan · 기능·디자인 변경 인터뷰**

이 웹앱에 기능을 추가하고 싶어.  
나를 인터뷰해서 원하는 기능과 화면을 구체화하고  
수정 계획부터 제안해줘. 내가 확인하면 구현하자.

AI가 제안한 계획에서 내 의도와 다른 부분을 고칩니다. 확인한 뒤 구현합니다.

# 인터뷰 기반 기능·디자인 변경

① 기획 프롬프트·검토

② /plan 입력 사진

③ 명령 설명 사진

/plan|



60m

Opus 5.5 Medium

⚡ Auto



입력 중인 사진입니다. 아래는 아직 Auto입니다. 모드 표시에서 Plan을 선택·확인하고 기획 프롬프트를 보냅니다.

# 인터뷰 기반 기능·디자인 변경

① 기획 프롬프트·검토

② /plan 입력 사진

③ 명령 설명 사진

이건 `~/.claude/commands/`에 있는 커스텀 커맨드입니다. 실행하면 **planner 에이전트**가 계획서를 만듭니다. 계획서에는 요구사항 정리, 단계별 계획, 리스크, 난이도가 들어갑니다. 그다음 사용자가 "yes"라고 답할 때까지 기다립니다.

```
/plan 결제 기능에 환불 처리 추가해줘
```

이렇게 뒤에 만들고 싶은 기능을 적으면 됩니다. 계획이 나오면 이렇게 답하시면 돼요.

- **yes** 또는 **진행해**: 구현을 시작합니다.
- **modify: 2단계는 빼줘**: 계획을 수정합니다.
- **different approach: ...**: 다른 방식으로 다시 계획합니다.

## 어떤 걸 쓸까?

| 상황                             | 추천                                       |
|--------------------------------|------------------------------------------|
| 평소 작업에서 "먼저 계획 보고 승인" 흐름을 원할 때 | 내장 Plan Mode ( <b>Shift+Tab</b> / 모드 선택) |
| 리스크와 난이도까지 정리된 상세 계획서가 필요할 때   | <b>/plan &lt;기능 설명&gt;</b>               |

해당 환경의 명령 설명입니다. 실제 PRD·IA 결과가 아닙니다. 수업은 내장 Plan 선택을 기본으로 합니다.

# grill-me: 인터뷰를 스킬로 재사용

스킬은 자주 쓰는 작업 방법을 저장해 이름으로 부르는 도구입니다.

① 무엇을 하나

② 전체 내용 설치

**name** · 부를 이름

name: grill-me

이름을 보고 /grill-me로 부릅니다.

**description** · 무엇을, 언제 할지

“잘 도와줘”보다 구체적으로:

기획 전에 인터뷰해서 빈틈을 찾기

**SKILL.md** 맨 위 예시

---

name: grill-me

description: 기획 전에 인터뷰로

빈틈을 찾을 때 사용한다.

disable-model-invocation: true

---

이 예시는 두 항목을 읽는 연습용입니다.

다음 탭에서 전체 스킬을 그대로 저장합니다.

수업용 스킬은 이름을 직접 불러 사용합니다.

# grill-me: 인터뷰를 스킬로 재사용

스킬은 자주 쓰는 작업 방법을 저장해 이름으로 부르는 도구입니다.

① 무엇을 하나

② 전체 내용 설치

## 기획의 빈틈을 질문으로 찾는 스킬

"누가 쓰나요?" "사진이 없으면요?"

만들기 전에 빠진 결정을 묻습니다.

### LV5 · 사진 메모 기획

사용 예: /grill-me로 이 웹앱의 기획을 점검해줘.

수업용 한국어판

원본: Matt Pocock · MIT

전체 SKILL.md + 저장 프롬프트

전체 SKILL.md와 저장 프롬프트는 다음 페이지부터 이어집니다.

내용을 생략하지 않고 모두 실었습니다.

# grill-me: 인터뷰를 스킬로 재사용 · 전체 내용 1

원본 슬라이드 45 · ② 전체 내용 설치 · 이어지는 내용을 모두 제공합니다.

아래 내용을 `.claude/skills/grill-me/SKILL.md`로 저장해줘.  
설치 후 파일 경로와 내용을 확인해줘.

```
```markdown
```

```
---
```

```
name: grill-me
```

```
description: 기획이나 설계를 코드로 만들기 전에 사용자를 집요하게 인터뷰해 빈틈을 찾는다. 사용자가 /grill-me 를 입력하거나 "기획을  
캐물어줘", "털어줘"라고 할 때 쓴다.
```

```
disable-model-invocation: true
```

```
---
```

```
# grill-me - 만들기 전에 기획 털기
```

```
> 교육용 한국어판. 원본: Matt Pocock `mattpocock/skills`의 `grill-me` + `grilling` (MIT License, 원문은 `_원  
본/mattpocock-skills/`).
```

```
> 바꾼 점:
```

```
> - 원본은 `grilling` 스킬을 다시 부르는데, 이 파일 하나로 동작하도록 합쳤다.
```

```
> - 비개발자 수업용으로 라운드당 질문 수를 줄였다.
```

# grill-me: 인터뷰를 스킴으로 재사용 · 전체 내용 2

원본 슬라이드 45 · ② 전체 내용 설치 · 이어지는 내용을 모두 제공합니다.

> - 하위 에이전트로 사실을 조사하는 부분을 "직접 파일 읽기"로 바꿨다.

사용자와 \*\*같은 그림을 그릴 때까지\*\* 인터뷰한다. 코드는 만들지 않는다.

## ## 진행 방식

1. 사용자가 준 기획(예: `@기획.md`)과 폴더 안 파일을 먼저 읽는다.
2. 기획을 \*\*결정 나무\*\*로 본다. 결정 하나에는 그 결정에 달린 다음 결정들이 가지처럼 붙어 있다.
3. \*\*라운드\*\* 단위로 묻는다.
  - 지금 바로 물을 수 있는 질문만 묻는다. 앞선 답이 정해져야 물을 수 있는 질문은 다음 라운드로 미룬다.
  - 한 라운드에 질문은 \*\*최대 3개\*\*다(사용자가 다르게 정하면 따른다).
  - 질문마다 번호를 붙이고 \*\*추천 답\*\*을 함께 준다.
  - 답을 받기 전에는 다음 라운드로 넘어가지 않는다.
4. 파일에서 확인할 수 있는 \*\*사실\*\*은 사용자에게 묻지 말고 직접 찾는다. 사용자에게는 \*\*결정\*\*만 묻는다.
5. 답이 쌓이면 나무를 다시 그리고 다음 라운드를 묻는다.
6. 빈 가지가 없거나, 사용자가 정한 질문 수에 도달하면 끝낸다.
  - 마지막에는 정해진 것, 아직 열린 것, 일부러 뺀 것을 요약한다.
  - 사용자가 요청하면 기획 파일을 고친다.

# grill-me: 인터뷰를 스킴으로 재사용 · 전체 내용 3

원본 슬라이드 45 · ② 전체 내용 설치 · 이어지는 내용을 모두 제공합니다.

– \*\*사용자가 "이해가 맞다"고 확인하기 전에는 아무것도 만들지 않는다.\*\*

## 라운드 형식

``

? \*\*Q1\*\* – \*\*<질문 제목>\*\*: <질문 내용. 보기가 있으면 보기도>

➡ 추천: <추천 답과 한 줄 이유>

---

? \*\*Q2\*\* – \*\*<질문 제목>\*\*: <질문 내용>

➡ 추천: <추천 답과 한 줄 이유>

``

## 잘 캐묻는 질문의 예

# grill-me: 인터뷰를 스킴로 재사용 · 전체 내용 4

원본 슬라이드 45 · ② 전체 내용 설치 · 이어지는 내용을 모두 제공합니다.

- **\*\*누가·어디서\*\***: 장갑 낀 손으로도 누를 수 있어야 하나요? 화면 크기는 폰인가요?
- **\*\*예외\*\***: 사진 없이 저장해도 되나요? 같은 설비를 하루 두 번 점검하면 어떻게 되나요?
- **\*\*경계\*\***: 이번 버전에서 일부러 안 하는 것은 무엇인가요(로그인, 서버, 공유)?
- **\*\*완료 기준\*\***: 무엇이 되면 "다 됐다"고 말할 수 있나요?

**##** 하지 않는 것

- 코드 작성, 파일 생성, 설치 - 사용자가 확인하기 전에는 하지 않는다.
- 한 번에 많은 질문을 쏟아내기.
- 사용자가 정할 결정을 대신 정해 버리기.
- ````

# LEVEL 5

## 사진으로 남기는 내 기록

이번 레벨에서 만드는 것  
사진과 메모를 저장하고, 나중에 다시 보기

도구  
VS Code + Claude  
Code

난이도  
★★★★☆

목표·인터뷰

내가 승인

구현

직접 확인·수정

# 사진 기록 앱에서 이렇게 활용합니다

사진과 메모를 남기고 다시 보는 기능을 브라우저로 만들 수 있습니다.

## ① 사진 남기기

사진 파일을 골라 올립니다.  
카메라를 연결하면  
바로 촬영할 수도 있습니다.

## ② 메모 적기

사진에 설명을 붙입니다.  
키보드 대신 음성 입력을  
더하는 방법도 있습니다.

## ③ 저장하고 다시 보기

기록을 브라우저에 저장해  
새로고침 뒤에도  
다시 볼 수 있게 만듭니다.

## 필요하면 이런 기능도 결합합니다

위치로 촬영 장소를 남기거나, 알림으로 기록을 잊지 않게 합니다.  
내게 필요한 기능과 저장 방법을 다음 인터뷰에서 선택합니다.

카메라·음성·위치·알림은 권한 허용 여부와 기기에 따라 다릅니다.

막히면 **사진 파일 선택** · **키보드 메모** · **장소 이름 입력**으로 기록을 이어갑니다.

# 기획 전에 고르는 화면 레퍼런스

① 찾기·캡처·첨부

② 참고할 부분 선택

원하는 화면을 찾아, 좋아하는 부분과 함께 AI에 전달합니다.

## 1. 화면 찾기

Pinterest · Behance

**photo journal UI**

**dashboard UI**로 검색

## 2. 필요한 부분 캡처

카드 · 메뉴 · 여백

마음에 드는 화면 한 장

출처 링크도 보관

## 3. 이미지 첨부

Claude Code 대화에

캡처를 첨부하고

원하는 부분을 설명

[Pinterest 검색 열기](#) · [Behance 검색 열기](#)

Windows: Win + Shift + S · Mac: Cmd + Shift + 4

# 기획 전에 고르는 화면 레퍼런스

① 찾기·캡처·첨부

② 참고할 부분 선택



[Journal app ui design · Busra Ozelikors · Behance](#)

## 인터뷰에 가져갈 참고

“이 이미지의 카드 배치와 여백이 마음에 들어요.”

적용하고 싶은 부분을 골라 둡니다.

다음 기획 인터뷰에 이미지와 함께 전달합니다.

예시: 카드 배치·둥근 모서리·여백을 참고합니다.

색과 내용은 내 웹앱에 맞게 정합니다.

# 내 사진 기록의 기획 인터뷰

## 무엇을 모으고 싶나요?

책상 물건 · 취미 기록 · 가상 전시  
내가 정한 소재로 시작합니다.

## 답하며 정할 것

사진과 메모를 입력하는 방법  
저장한 기록을 다시 보는 화면

### Plan · grill-me로 기획

[내 사진 기록 주제]로 웹앱을 만들고 싶어.  
사진·메모를 저장하고 나중에 다시 보고 싶어.  
첨부한 레퍼런스의 [마음에 드는 부분]도 참고하자.  
/grill-me로 한 번에 질문 하나씩 나를 인터뷰해서 필요한 기능을 정리하자.  
정리한 기능을 바탕으로 IA를 작성해줘.  
IA를 내가 검토·승인하면 추가 인터뷰로 저장 방법과 동작을 정해 PRD를 구체화하자.  
내가 PRD를 검토·승인하면 개발 계획을 세우자.

# 확인한 기획으로 사진 기록 구현

## 기획에서 구현으로

1. 5-0의 기획·개발 계획 확인
2. 기획대로 개발 → 실행 승인
3. 로컬 서버에서 웹앱 열기

같은 vibe-lab의 memo/에서 이어갑니다.  
직접 찍은 소품 사진이나 교육용 이미지를 사용합니다.

## 내가 직접 확인

1. 사진 선택 → 메모 입력 → 저장
2. 목록에서 방금 저장한 카드 확인
3. 새로고침 → 사진·메모 유지
4. 폰 크기에서 입력·저장 버튼 확인

**F12** → 폰 아이콘 · 가로 넘침도 확인

날씨·사진 분석·스마트폰 터널은 뒤쪽 선택 자료에서 필요할 때 이어갑니다.

# LEVEL 6

## 나만의 CSV 분석 서비스

이번 레벨에서 만드는 것

가상 CSV 선택·준비 → 드래그앤드롭 → Claude 분석·결과 확인

도구  
VS Code + Claude  
Code

난이도  
★★★★☆

목표·인터뷰

내가 승인

구현

직접 확인·수정

# CSV: 표 데이터를 담는 파일

① 파일과 기록

② 항목과 분석 질문

sales.csv는 7일 동안 두 지역·두 제품의 판매를 담은 가상 데이터입니다.

28행 · 한 행은 날짜·지역·품목별 합계

```
날짜,지역,품목,판매량_톤,단가_원_톤,매출_원
2026-10-01,A,제품1,60,500000,30000000
2026-10-01,A,제품2,70,550000,38500000
```

첫 줄은 열 이름, 다음 줄부터 기록입니다.

이 두 행에서 읽을 수 있는 것

지역 A의 10월 1일 판매량: **130톤**

매출: **68,500,000원**

매출 = 판매량 × 톤당 단가

판매량과 매출의 단위를 구분합니다.

실제 회사 가격·실적이 아닙니다.

학생용 ZIP의 sales.csv와 CSV\_활용안내.md에 전체 데이터·열 설명·검산 기준을 넣었습니다.

# CSV: 표 데이터를 담는 파일

① 파일과 기록

② 항목과 분석 질문

## 분석에 필요한 여섯 열

날짜 · 지역 · 품목

판매량(톤) · 톤당 단가(원/톤) · 매출(원)

날짜별 변화, 지역별 비교,  
제품별 비중을 확인할 수 있습니다.

## 예: 어떤 지역의 판매가 늘었나?

지역별·날짜별 판매량을 비교합니다.

큰 변화가 보이면 해당 행을 확인합니다.

변화의 원인은 메모나 추가 자료로 확인합니다.

참고 데이터로 연습하거나, 내 질문에 맞는 CSV를 인터뷰로 기획해 새로 생성합니다.

# 분석 서비스의 **사용자와 질문**

누가 어떤 질문으로 CSV를 분석할지 먼저 정합니다.

## 누가 사용하나

팀장 · 동료 · 나  
서비스를 쓰는 사람의  
궁금한 점

## 무엇을 알고 싶나

판매 변화 · 재고 현황  
지역 비교 · 점검 결과  
핵심 질문 하나

## 어떻게 보여주나

짧은 요약과 핵심 수치  
표 또는 차트  
필요한 부분부터

# 내 분석에 쓸 가상 CSV 준비

앞에서 정한 사용자와 분석 질문을 이어서 씁니다. 아래에서 한 가지를 선택합니다.

① 제공 CSV 사용

② 새 CSV 기획·생성

Plan · sales.csv를 @로 지정

앞에서 정한 사용자와 분석 질문에 이 sales.csv를 쓰고 싶어.  
/grill-me로 한 번에 질문 하나씩, 이 데이터로 질문에 답할 수 있는지 확인하자.  
이미 있는 항목·단위·기간을 읽고, 부족한 정보가 있으면 알려줘.  
내가 확인하면 이 CSV로 결과 화면을 기획하자.

## 선택하거나 생성한 파일을 직접 읽기

항목 이름 · 단위 · 기간 · 몇 개의 값을 확인합니다.

# 내 분석에 쓸 가상 CSV 준비

앞에서 정한 사용자와 분석 질문을 이어서 씁니다. 아래에서 한 가지를 선택합니다.

① 제공 CSV 사용

② 새 CSV 기획·생성

Plan · grill-me로 새 데이터 기획

내가 정한 분석 질문에 맞는 가상 CSV를 만들고 싶어.

/grill-me로 필요한 항목과 단위, 기간을 한 번에 하나씩 물어보며 데이터를 기획하자.

데이터 기획을 보여줘. 내가 검토·승인하면 그 기획대로 CSV 파일을 만들어줘.

## 선택하거나 생성한 파일을 직접 읽기

항목 이름 · 단위 · 기간 · 몇 개의 값을 확인합니다.

# CSV 분석 서비스의 결과 화면 기획

① 서비스 기획·IA·PRD

② 서비스 흐름 참고

Plan · CSV를 @로 지정한 뒤

앞에서 정한 사용자·분석 질문과 준비한 CSV를 이어서 사용하자.  
파일을 드래그앤드롭하면 API가 분석하고, 결과를 대시보드로 보여주는 서비스를 기획하고 싶어.  
/grill-me로 한 번에 질문 하나씩, 필요한 요약·표·차트·필터와 화면 흐름을 정해 IA를 작성해줘.  
이미 정한 내용은 다시 묻지 말고, 빠진 조건만 확인하자.  
IA를 내가 승인하면 파일 형식·비교 기준·저장·분석 실패 시 동작을 더 물어 PRD를 구체화하자.  
내가 PRD를 검토·승인하면 API 연동을 포함한 구현 계획을 세우자.

내 파일의 어떤 데이터를 어떻게 볼지 정합니다. AI가 정한 화면과 조건을 읽고 고친 뒤 승인합니다.

# CSV 분석 서비스의 결과 화면 기획

① 서비스 기획·IA·PRD

② 서비스 흐름 참고

## 파일 넣기

CSV · 엑셀 드롭



## 데이터 읽기

시트 · 열 · 단위 확인



## API 분석

질문 · 비교 기준



## 내 대시보드

요약 · 표 · 차트

### 시간에 따른 변화

날짜별 판매량이  
늘었는지 줄었는지



### 항목끼리 비교

지역 A와 B 중  
어디의 판매량이 더 큰지

A

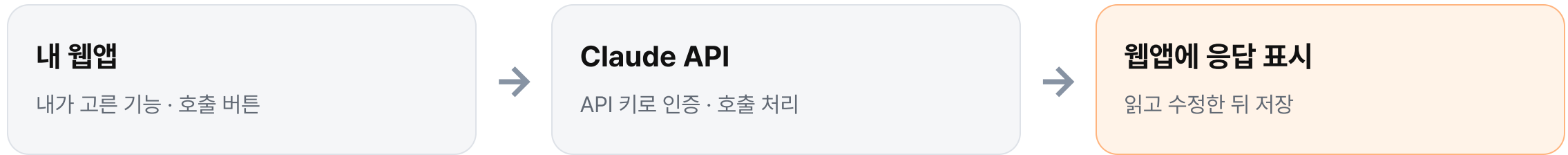
100톤

B

150톤

# 웹앱의 Claude API 호출

웹앱에서 보낸 내용을 Claude가 처리해 응답합니다.



# 승인한 CSV 분석 서비스의 구현과 확인

앞에서 정한 IA·PRD·분석 프롬프트대로 만듭니다.

구현 모드 · 계획 승인 후

승인한 기획과 분석 프롬프트대로 CSV 분석 서비스를 구현하자.

기획에서 정한 파일 형식의 드래그앤드롭과 Claude API 분석, 대시보드를 연결하고 로컬에서 실행해줘.

작게 나눠 첫 단계부터 구현하고 내가 확인하면 다음으로 넘어가자.

## 내가 직접 확인

파일 넣기 → 데이터 읽기 → API 분석 → 대시보드 표시  
원본의 숫자·단위와 분석 결과를 비교합니다.

# 값을 바꿔 원본과 다시 비교

같은 앱 · 갱신과 재확인

CSV의 값 하나를 바꿔 다시 넣으면 분석 결과도 갱신되게 하자.  
기획과 다른 부분이 있으면 수정 계획부터 보여줘. 내가 확인하면 고치자.

설명용 예시 · 변경 전

A 100톤 + B 150톤  
총 250톤

변경 후 직접 대조

B를 200톤으로 바꾸면 총 300톤  
표 · 차트 · 요약이 함께 바뀌나요?

# LEVEL 7

## CSV 분석 서비스의 Work OS 확장

이번 레벨에서 만드는 것

CSV 분석 서비스 + 업무 메모 → 할 일 분해·관리 → QA·전달

도구

Claude Code + Plan +  
점검 프롬프트

난이도



목표·인터뷰

내가 승인

구현

직접 확인·수정

# 같은 CSV 서비스에 메모·할 일 더하기

## 실습 7-1 · 확장 인터뷰

Plan · LV6 앱을 이어서

내 CSV 분석 서비스에 메모와 할 일 관리 기능을 더해  
나만의 Work OS로 확장하고 싶어.  
/grill-me로 한 번에 질문 하나씩 나를 인터뷰해서 필요한 기능을 정리해줘.  
각 기능이 어떤 불편을 해결하는지와 우선순위를 함께 정하자.  
먼저 기능 목록과 이번에 만들 범위를 내가 확인하게 보여줘.

이번에는 기능과 범위부터 정합니다. 다음 페이지에서 화면과 동작 조건을 구체화합니다.

# 정한 기능의 UI/UX와 IA

① 화면·동작 기획

② 흐름 참고

Plan · 앞에서 확인한 기능 목록을 이어서

방금 정한 Work OS 기능을 바탕으로 UI/UX와 화면 IA를 기획하자.

기존 CSV 분석 화면과 새 기능을 어떻게 연결할지 한 번에 질문 하나씩 인터뷰해줘.

IA를 내가 검토·승인하면 저장할 내용·진행 상태·다시 보는 방법을 더 물어 PRD를 구체화하자.

내가 PRD를 수정·승인하면 개발 계획을 세우자.

기능은 다시 처음부터 정하지 않습니다. 화면과 흐름을 확인한 뒤 동작·저장 조건으로 이어갑니다.

# 정한 기능의 UI/UX와 IA

① 화면·동작 기획

② 흐름 참고

가능한 흐름의 예시입니다. 내 기능과 다르면 바꿉니다.

## ① CSV 입력

분석할 데이터  
입력 자료 확인

## ② 메모 더하기

회의·업무 맥락  
내 목적과 기준

## ③ Claude 업무 정리

분석과 메모 연결  
할 일·우선순위 제안

## ④ 할 일 관리

제안 확인·수정  
저장·진행·완료

한 가지 가능한 구성입니다. 필요한 화면과 기능은 인터뷰로 정합니다.

# Work OS의 단계별 구현과 로컬 확인

## 실습 7-2

확인한 기획과 개발 계획대로 구현합니다.

### 단계별 구현·로컬 확인

승인한 기획대로 기존 CSV 분석 서비스를 확장하자.  
개발 단계를 작게 나누어 단계별로 구현해줘.  
각 단계가 끝나면 로컬에서 실행하고, 내가 확인할 화면과 동작을 알려줘.  
확인하면서 발견한 문제는 다음 단계에 반영하자.

#### 핵심 기능

해결하려던 불편이  
줄었는지 확인합니다.

#### 화면과 동작

기획한 흐름대로  
직접 눌러 봅니다.

#### 피드백

원하는 모습과 다른  
부분을 수정할 프롬프트를 보냅니다.

# 웹앱의 QA 리스트와 수정

## 실습 7-3

QA는 만든 웹앱이 기대한 대로 작동하는지 확인하는 일입니다.

① QA 리스트

② 수정 인터뷰

실습 프롬프트

승인한 IA·PRD를 옆에 두고 이 웹앱을 확인할 QA 리스트를 만들어줘.

입력 → 분석 → 메모 → 할 일 저장 → 새로그침 흐름에서

무엇을 누르면 어떤 결과가 나와야 하는지 짧게 알려줘.

리스트를 보며 직접 눌러 보고 다른 부분을 수정할 프롬프트를 보냅니다.

# 웹앱의 QA 리스트와 수정

## 실습 7-3

QA는 만든 웹앱이 기대한 대로 작동하는지 확인하는 일입니다.

① QA 리스트

② 수정 인터뷰

실습 프롬프트

지금은 [현재 모습]인데 [원하는 모습]으로 바꾸고 싶어.  
더 필요한 정보가 있으면 내게 물어보고 수정 계획부터 세우자.  
내가 확인하면 수정하고 다시 점검하자.

리스트를 보며 직접 눌러 보고 다른 부분을 수정할 프롬프트를 보냅니다.

# 공유 전 **보안 확인**

공유하기 전에 키와 데이터 접근 범위를 확인합니다.

## API 키 보호

코드·GitHub·공유 ZIP에  
키를 넣지 않습니다.  
공개 서비스의 키는  
서버에서 관리합니다.

## 자료·권한 확인

회사·개인 자료 사용 규칙  
로그인과 접근 권한  
다른 사람의 기록이  
보이지 않는지 확인

## 외부 입력 주의

CSV·메모·외부 문서는  
분석할 자료입니다.  
그 안의 명령을 그대로  
실행하지 않습니다.

AI에게 점검을 맡길 때

공유하기 전에 API 키 노출과 데이터 접근 권한을 점검해줘.  
위험한 부분과 내가 직접 확인할 항목을 알려줘.

[더 읽기 · 요즘 IT '바이브코더'를 위한 기초 보안 A to Z](#)

# 웹앱의 압축과 전달

## 실습 7-4

실습 프롬프트

이 웹앱을 동료에게 전달할 수 있게 압축해줘.  
로컬 실행 방법도 함께 넣어줘.

API 키는 외부로 유출되면 안 됩니다.

**압축**  
필요한 파일 모으기



**전달**  
실행 방법 함께



**확인**  
압축을 풀고 실행

# 선택 자료 · 필요한 것만 골라 보기

핵심 실습을 마쳤습니다. 필요한 확장만 보고 오늘의 점검과 2주 과제로 마무리합니다.

## AI와 작업 환경

설계 트렌드 · MCP  
스킬 · 서브에이전트

## 사진 앱 확장

Claude 사진 분석 · 날씨 API  
터널링으로 폰에서 열기

## 계속 쓰는 방법

하네스 · Phase 개발  
보고서 · Playwright

필요한 자료만 골라 보세요. [오늘 배운 것 점검과 2주 과제로 바로 이동 →](#)

# Gemini의 앱 연결과 이미지 활용

① 구글 생태계

② Nano Banana

Gemini 앱은 익숙한 구글 서비스와 연결해 쓰기 편합니다.

## 메일 · 문서 · 일정

Gmail 메일 요약

Drive 자료 찾기

Calendar 일정과 연결

## 이렇게 활용

“메일에서 회의 내용을 찾아줘.”

“관련 문서도 함께 정리해줘.”

자료를 옮겨 붙이는 일을 줄입니다.

연결할 앱은 내가 선택합니다. 계정·권한에 따라 사용할 기능이 다릅니다.

[Google 도움말 · 연결된 앱](#)

# Gemini의 앱 연결과 이미지 활용

① 구글 생태계

② Nano Banana

## Nano Banana: 사진을 유지하며 바꾸는 이미지 생성·편집

### 왜 주목받았나

인물의 모습을 유지하며  
배경 · 의상 · 스타일 변경  
여러 사진을 자연스럽게 합치기

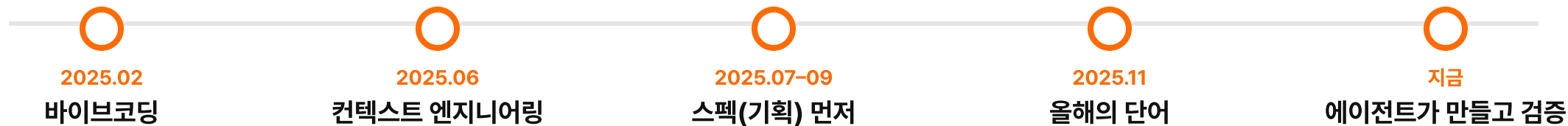
### 업무에 쓰는 예

제품 사진의 배경 바꾸기  
시안과 홍보 이미지 만들기  
Pro는 이미지 속 글자 표현도 강화

2025년 공개 당시 구글은 이미지 편집 평가 1위로 소개했습니다.

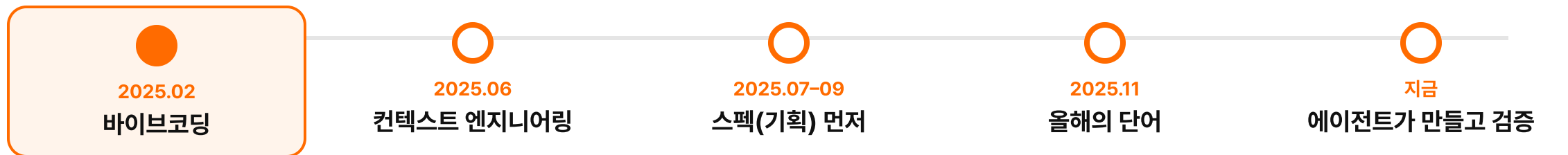
[Google](#) · [Nano Banana](#) · [Nano Banana Pro](#)

## 지난 1년의 변화: **기획과 검증**



연표는 참고입니다. 오늘은 한 줄 제작 → 기획 → 직접 확인의 변화를 봅니다.

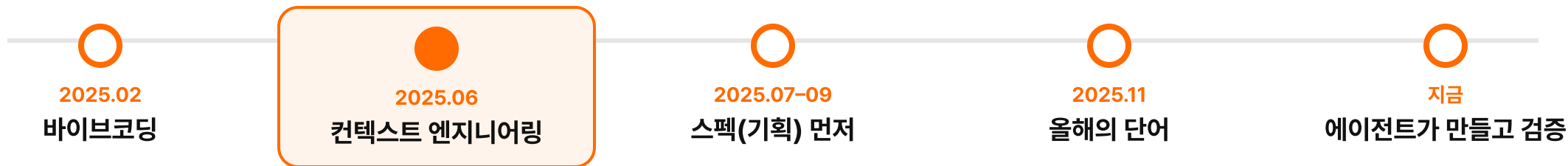
## 지난 1년의 변화: **기획과 검증**



**"코드는 잊고 느낌대로 시킨다"**

말로 만들고 결과를 보며 고칩니다. → LV1-2 게임

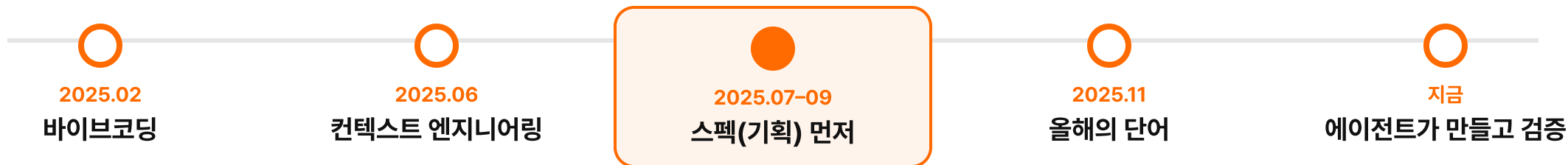
## 지난 1년의 변화: **기획과 검증**



### "프롬프트보다 맥락"

필요한 조건과 자료를 함께 줍니다. → **LV3 기획·레퍼런스**

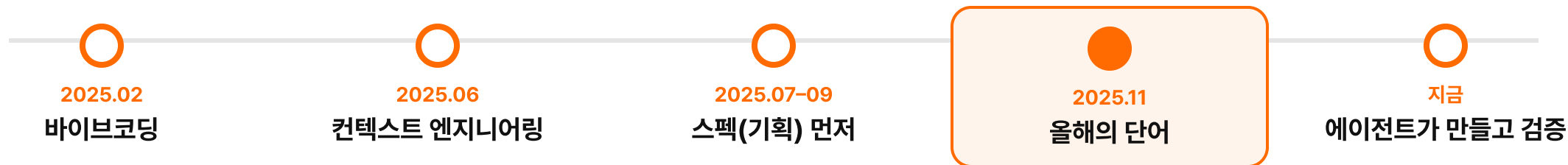
## 지난 1년의 변화: **기획과 검증**



### "막 시키지 말고 기획서부터"

만들 기능과 완료 기준을 먼저 정합니다. → LV3 기획 → LV4 계획·승인

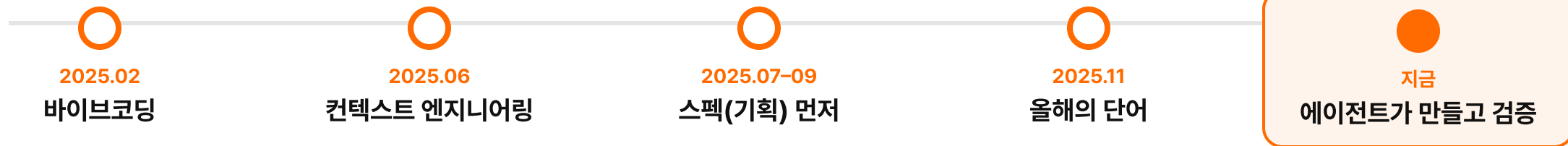
## 지난 1년의 변화: **기획과 검증**



**Collins 사전 올해의 단어 = vibe coding**

말로 만드는 방식을 가리키는 용어가 널리 알려졌습니다.

## 지난 1년의 변화: **기획과 검증**



**계획 → 구현 → 테스트까지 에이전트가**

사람이 완료 기준을 정하고 직접 실행해 확인합니다. → **LV7 점검**

# AI 작업 설계 범위의 확장

프롬프트에서 시작해 정보·환경·작업 흐름까지 설계합니다.

## 프롬프트

프롬프트에 무엇을 담을  
까?

목표·결과·조건

## 컨텍스트

무엇을 알려 줄까?

관련 자료·예시·최신 결정

## 하네스

어떻게 일하게 할까?

도구·규칙·검증

## 그래프

어떤 흐름으로 할까?

순서·분기·동시 작업

프롬프트 → 정보 → 환경 → 흐름 · 함께 사용하는 네 가지 관점

# 프롬프트 엔지니어링: 좋은 프롬프트의 구성

목표·맥락·결과 형태·조건을 주면 의도가 더 분명해집니다.

## ① 목표 · 무엇을 할까?

누구를 위해, 어떤 결과가 필요한가

## ② 맥락 · 무엇을 참고할까?

자료·현재 상황·원하는 예시

## ③ 결과 형태 · 어떻게 받을까?

표·보고서·웹앱 등 원하는 형식

## ④ 조건 · 무엇을 지킬까?

포함할 것·제외할 것·확인 기준

구성을 이해하기 위한 예시

이 매출 CSV로 팀장에게 공유할 보고서를 작성해줘.  
지난달과 이번 달을 비교하고, 변화가 큰 항목을 정리  
해줘.

금액 단위는 만원이야. 표와 짧은 설명으로 보여줘.  
자료로 알 수 없는 원인은 추측하지 말고, 확인할 질  
문으로 남겨줘.

이 조건을 처음부터 다 쓰기 어렵다면?  
의도부터 설명하고 메타프롬프팅으로 함께 정합니다.

# 하네스: 환경·범위·적용

① 작업 환경

② 전역·프로젝트

③ 충돌과 우선순위

④ 내 프로젝트에 적용

AI가 작업하고 결과를 확인할 환경과 절차를 구성합니다.

## 도구와 공간

프로젝트 파일 읽기·수정  
브라우저 실행·화면 점검

## 규칙과 역할

CLAUDE.md의 공통 규칙  
스킬의 절차·담당 에이전트

## 검증과 기록

기대 결과·실제 테스트  
진행 상황·미해결 내용

## 보고서 작업의 예

파일 읽기 → 보고서 작성 → 수치 대조 → 확인 결과 기록

한 번의 답변을 넘어, 다음 작업에서도 확인한 절차를 이어 갑니다.

선택 자료 · 시연·복습

# 하네스: 환경·범위·적용

① 작업 환경

② 전역·프로젝트

③ 충돌과 우선순위

④ 내 프로젝트에 적용

전역은 기본법, 프로젝트는 그 업무에 맞춘 특별규칙으로 생각합니다.

## Global use · 모든 프로젝트의 공통 기준

한국어 답변 · 결과 확인 습관

자주 쓰는 공통 스킬과 도구

예: ~/.claude/CLAUDE.md

## Project use · 이 프로젝트의 세부 기준

보고서 양식 · 계산 기준 · 확인 방법

이 프로젝트에서 쓸 스킬과 도구

예: 프로젝트/CLAUDE.md

전역은 공통 기본값, 프로젝트는 이 업무의 구체적 기준입니다.

예: 공통은 한국어 답변, 이 프로젝트의 고객 안내문은 영어로 작성.

# 하네스: 환경·범위·적용

① 작업 환경

② 전역·프로젝트

③ 충돌과 우선순위

④ 내 프로젝트에 적용

우선순위는 프로그램 설정과 AI가 읽는 지침을 나눠 봅니다.

## 설정 · 정해진 적용 순서

같은 일반 설정이면 **프로젝트 > 전역**  
조직의 강제 설정은 이보다 우선  
법의 **강행규정**처럼 임의 변경 불가

## CLAUDE.md · 프로젝트 예외를 명시

두 파일을 함께 읽고 프로젝트 지침을 나중에 읽음  
**“이 프로젝트의 고객 안내문은 영어로”**  
충돌은 자동 덮어쓰기를 기대하지 말고 정리

일반법에 특별규칙을 더하듯, 프로젝트 예외를 구체적으로 씁니다.

두 CLAUDE.md의 충돌이 자동으로 해결되는 것은 아닙니다. [공식 문서](#)

# 하네스: 환경·범위·적용

① 작업 환경

② 전역·프로젝트

③ 충돌과 우선순위

④ 내 프로젝트에 적용

이미 만든 규칙과 스킬에 작업 순서·확인 방법을 연결합니다.

Plan · 현재 프로젝트에서 구성

이 프로젝트에 맞는 하네스를 간단히 구성하고 싶어.

기존 규칙과 스킬을 보고 나를 인터뷰해서 작업 방식과 확인 방법을 정리해줘.

내가 확인하면 프로젝트에 적용하고, 작은 작업으로 점검해줘.

## ① 규칙과 범위

CLAUDE.md 검토

전역 지침과 충돌 확인

## ② 작업 방법

필요한 스킬 선택

단계별 순서 정리

## ③ 실행과 확인

작은 변경 직접 확인

결과와 다음 할 일 기록

# MCP와 커넥터: AI와 도구 연결

대화에 복사해 넣는 대신, AI가 연결된 도구에서 자료를 읽고 작업합니다.

## MCP · 공통 연결 규격

AI와 외부 도구·데이터를  
연결하는 표준 방식.

**AI용 USB-C**에 비유할 수 있습니다.

## 커넥터 · 제품의 연결 기능

Claude Desktop에서는  
**커넥터**로 도구 연결을 관리.

예: Notion 문서 연결

### Claude Desktop

AI와 대화



### 커넥터 · MCP 연결

계정·권한 확인



### 문서·도구

허용된 자료·기능 이용

# AI 활용 기능의 역할 구분

| 개념        | 맡는 역할               | 보고서 작업의 예             |
|-----------|---------------------|-----------------------|
| 모델        | 이해하고 판단·생성하는 AI     | 자료를 읽고 보고서 초안 작성      |
| 도구 · API  | 실행 기능 · 프로그램의 연결 창구 | 파일 읽기 · 계산 · 데이터 가져오기 |
| MCP · 커넥터 | 도구·데이터를 AI에 연결      | 허용된 문서 서비스 연결         |
| 스킬        | 반복 작업의 방법과 기준       | 우리 팀의 보고서 작성 절차       |
| 에이전트      | 목표를 따라 도구로 작업 수행    | 자료 수집 → 작성 → 점검       |
| 플러그인      | 기능을 묶어 추가하는 패키지     | 스킬·도구 연결 등을 함께 설치     |

스킬은 일하는 방법, 도구는 실행 기능, 에이전트는 작업 수행자입니다.

# 실습에서 쓰는 용어 지도

지금: 프롬프트 = AI에게 주는 지시 · 뒤의 말: 해당 레벨에서 뜻과 시연을 함께 봅니다.

시지 크기보 제이A

토으스형 변이, 제형이, 을것무

제형변 이어날 을형문 크프롬프 은중

"제형은중 변이시 제형이"  
(0.8.5VJ)

중 형 은프 제형변 을것무

(8VJ) 첫 형 은.능지 삼형.제.제구

제형변 크투크 을형제 제형

제형변형 형 을형삼.을형.제폴  
(4VJ)

제형 크기 제형제

(4VJ) 은변 제 을것불.수삼.제형제

제형형제 제형제 제

제형제 제형 제 = jeonllcol  
(4VJ) 소주

두형 크형제주 제형제제제

제형제 제형이 제형제제 제형이"  
(5VJ) "제

호변제제 은제 제형제제

제형제 .제 제형제 제형제제.제형  
(5VJ) 제형제제 제형 제형제제 제형

# 실습에서 쓰는 용어 지도

**지금:** 프롬프트 = AI에게 주는 지시 · **뒤의 말:** 해당 레벨에서 뜻과 시연을 함께 봅니다.

## AI에게 보내는 지시

무엇을, 어떻게, 어떤 형식으로

## 좋은 프롬프트 문장을 같이 만들기

"어떻게 시키면 좋을까?"  
(LV2·3·6)

## 무엇을 만들지 적은 한 장

누가·왜·핵심 기능·안 할 것 (LV3)

## 웹앱 파일을 다루는 작업대

폴더·파일·실행을 한 화면에서  
(LV4)

## 화면과 버튼 동작

게임판·점수·눌렀을 때 반응 (LV4)

## 내 PC에서 실행하기

localhost = 이 PC를 가리키는  
주소 (LV4)

## 프로그램끼리 주고받는 창구

"이렇게 호출하면 이렇게 응답할  
게" (LV7)

## 법인카드 같은 비밀번호

인증·사용량에 연결된 값. 공유하  
거나 코드에 넣지 않습니다 (LV7)

# 개념 확인 퀴즈

Q1. AI가 "다 만들었습니다"라고 했다. 다음 할 일은?

믿고 공유한다

직접 실행해서 확인한다

코드가 길면 완성됐다고 판단한다

대화를 지운다

Q2. 긴 대화에서 처음의 조건이 빠질 수 있는 이유는?

모델의 학습 데이터가 바뀌어서

인터넷이 느려서

다룰 수 있는 맥락에 한계가 있어서

규칙이 틀려서

# 개념 확인 퀴즈

Q1. AI가 "다 만들었습니다"라고 했다. 다음 할 일은?

믿고 공유한다

직접 실행해서 확인한다

코드가 길면 완성됐다고 판단한다

대화를 지운다

틀린 답도 그럴듯하게 보일 수 있습니다. 실행해서 확인이 사람의 일. LV7은 점검표를 따라 직접 눌러 확인합니다.

Q2. 긴 대화에서 처음의 조건이 빠질 수 있는 이유는?

모델의 학습 데이터가 바뀌어서

인터넷이 느려서

다룰 수 있는 맥락에 한계가 있어서

규칙이 틀려서

긴 대화는 요약되며 세부 조건이 빠질 수 있습니다. 새 대화에 목적을 다시 주고 반복 규칙은 파일로 둡니다.

# 반박 전후의 응답 비교

① 기본 응답 예시

② 반박 이후 예시

③ 사진 웹앱 사례

유튜브 링크를 넣으면 그 영상 내용을 일주일치 스레드(Threads) 게시글로 잘라서 만들어주는 월 9,900원 구독 서비스들 만들려고 해. 어때?

아이디어 설명 · “어때?”

좋은 문제의식이예요. “영상 하나 → 일주일치 스레드”는 크리에이터의 멀티플랫폼 부담을 실제로 줄여주고, 스레드 특성상 짧고 잦은 포스팅이 유리하니 궁합도 나쁘지 않습니다.

다만 시작 전에 확실히 해둘 게 하나 있어요: 본인 소유 영상만 대상으로 하는 건지, 아무 유튜브 링크나 넣을 수 있게 할 건지입니다. 후자면 타인 콘텐츠 재가공이라 저작권 리스크가 커지고, 전자(크리에이터가 자기 영상을 재활용)면 훨씬 안전하고 포지셔닝도 명확해져요.

가격(W9,900)은 진입장벽은 낮지만 CAC 회수를 위해선 불륨이 필요하니, 자동화 구축 전에 “이 자동화가 실제로 주당 몇 시간을 아껴주는가”를 소수 크리에이터 대상 수동 MVP로 먼저 검증해보는 걸 [사이드 채팅 시작](#) [답글](#)

원하시면 린 캔버스나 가설 검증 실험 설계부터 잡아드릴 수도 있고, [바로 MVP 스펙/아키텍처로 들어갈 수도 있어요](#) — 어느 쪽으로 갈까요?

답 · < > · 21시간 전

기본 응답 · 긍정적인 의견과 검토 조건

이 답으로 바로 만들까요?

좋아 보인다는 답만으로  
실제 수요와 차별성이  
확인된 것은 아닙니다.

다음 탭에서 같은 아이디어를  
비판적으로 검토하게 합니다.

# 반박 전후의 응답 비교

① 기본 응답 예시

② 반박 이후 예시

③ 사진 웹앱 사례

너는 이 아이디어에 투자하면 손해 보는 심사역이야. 치명적인 결함이 있는지 하나씩 체크하고, 근거를 들어 비판적으로 분석해줘. 괜찮다고 얼버무리지 말고 정말 위험한 지점만 3가지로 압축해줘.

자동 +

Sonnet 5   엑스트라

## 반론 하나에 내 답변

“기존 도구 대신  
이 웹앱을 쓸 이유는?”

내 근거와 생각을 설명하고  
다시 검토하게 합니다.  
반론도 사실인지 따져 봅니다.

같은 아이디어에 반박을 요구하는 프롬프트

투자 안 합니다. 세 가지 이유입니다.

**1. 이 제품은 “모트가 없는 LLM 레퍼”다 — 상품 자체가 무료 대체재로 즉시 복제된다**

핵심 기능이 “유튜브 자막 추출 → LLM 프롬프트로 스레드 포맷 변환”이 전부라면, 사용자가 자막을 직접 복사해서 ChatGPT/Claude에 “이거 스레드 7개로 잘라줘”라고 치면 동일한 품질의 결과를 무료로, 지금 당장 얻습니다. 이걸 가정이 아니라 이미 검증된 패턴입니다 — “GPT wrapper” 카테고리 스타트업들이 반복적으로 죽는 이유가 바로 이거예요. 진입장벽이 프롬프트 한 줄 수준이면, 카피캣이 하루 만에 같은 제품을 만들 수 있고 사용자는 굳이 월 구독료를 낼 이유가 없습니다. “왜 이 회사가 5년 뒤에도 존재해야 하는가”에 답을 못 하면 투자 불가 사유 1순위입니다.

**2. “생성”과 “발행” 사이에 실행 갭이 있고, 그 갭을 메우는 쪽(자동발행)은 통제 밖의 리스크다**

전파 시간 절약 가치는 “글을 만들어주는 것”이 아니라 “매일 알아서 올려주는 것”에서 나옵니다. 그런데 자동 포스팅을 하려면 Threads/Meta의 API 정책, 자동화 스크립트, 레이트리밋, 계정 정지 리스크를 떠안아야 합니다. 이런 플랫폼이 정책 한 줄만 바뀌도 서비스 전체가 죽는 구조적 리스크(플랫폼 종속 리스크)예요. 반대로 자동발행 없이 “텍스트만” 준다면, 사용자는 결국 매일 직접 복사·붙여넣기를 해야 하는데, 그럴 거면 애초에 무료인 1번(ChatGPT)을 씁니다. 둘 중 어느 쪽을 선택해도 사업이 취약해지는 딜레마 구조입니다.

**3. 타겟이 “니치의 니치”이고, 저가 구독 특유의 이탈률이 CAC 회수를 막는다**

이 서비스가 필요한 사람은 “유튜브를 꾸준히 올리면서 + 스레드도 진지하게 운영하고 + 그 반복작업을 자동화하고 싶은” 사람인데, 이 교집합 자체가 매우 작습니다. 게다가 콘텐츠 제작은 사용 패턴이 스파이크해서(이번 주 영상 못 찍으면 그 주는 서비스 쓸 일이 없음) 자발적 해지율이 구조적으로 높고, 9,900원 저가 구독은 카드 만료 등으로 인한 비자발적 이탈까지 겹칩니다. 개인용 저가 SaaS는 통상 월 5~10% 이탈이 흔한데, 이 정도 규모의 좁은 시장에서 그 이탈률을 상쇄할 만큼 저렴한 고객 확보 채널이 안 보입니다. “어디서 이 사람들을 싸게 모을 것인가”에 구체적인 답이 없으면 유닛 이코노믹스가 성립하지 않습니다.

반론 예시 · 차별성 / 실제 실행 / 사용자·구독 유지

# 반박 전후의 응답 비교

## ① 기본 응답 예시

## ② 반박 이후 예시

## ③ 사진 웹앱 사례

내 아이디어는 사진을 넣고 기록하고 LLM이 사진을 피드백해주는 웹앱이야.  
약한 가정과 실패할 이유를 들어 반박해줘.  
중요한 반론부터 하나씩 물어봐. 내 답을 듣고 다시 검토해줘.

Thought for 1s

5:46 PM

### ● 가장 큰 반론: 이건 문제가 아니라 기능 설명입니다.

"사진을 넣고, 기록하고, LLM이 피드백한다"는 ChatGPT나 Claude에 사진을 올리고 "이거 어때?" 하고 묻는 것과 차이가 없습니다. 이미 모든 사람의 폰에 들어 있는 기능이라서 웹앱을 따로 열 이유가 약합니다. 이 앱이 살아남으려면 범용 챗봇이 못 하는 것이 있어야 합니다. 후보는 세 가지입니다.

1. **특정 도메인에 맞춘 피드백.** 예를 들어 식단, 운동 자세, 피부, 그림 연습, 인테리어, 사진 구도 같은 영역입니다. 판단 기준과 프롬프트가 그 영역에 맞춰져 있어야 합니다.
2. **시간 축 기록.** "3주 전보다 나아졌다"처럼 누적된 사진을 비교하는 것입니다. 챗봇은 대화가 끊기면 이걸 못 합니다.
3. **습관 유지.** 기록을 계속하게 만드는 장치입니다.

지금 설명만으로는 셋 중 무엇인지 알 수 없고, 어느 쪽이냐에 따라 제품이 완전히 달라집니다. 도메인 특화가 아니라면 "그냥 ChatGPT 쓰면 되는데요"라는 반응을 이길 수 없습니다.

**질문: 어떤 사람이, 어떤 종류의 사진을, 무엇을 얻으려고 올리나요?** 가능한 실제로 이런 앱을 필요로 할 만한 사람 한 명 (본인이어도 됩니다)을 떠올려서, 그 사람이 지금은 이 문제를 어떻게 해결하고 있는지도 같이 말해 주세요.



실제 대화 예시 · 사진 기록 웹앱 아이디어에 대한 반론

## ① 기능 설명 → 해결할 문제

사진·기록·피드백만으로는  
별도 웹앱이 필요한 이유가 부족

## ② 기존 도구와의 차이

특정 분야의 피드백?  
누적 사진 비교? 기록 습관?

## ③ 내가 답할 질문

누가, 어떤 사진을,  
무엇을 얻으려고 올리나요?

# 반복적인 일은 **스킬화시키자**

스킬은 AI가 따라 할 작업 절차를 저장해 재사용하는 방법입니다.

## 반복하는 프롬프트

매번 같은 보고서 작성  
같은 항목으로 웹앱 점검

## SKILL.md로 정리

언제 쓰는지  
작업 순서·확인 기준

## 다음 작업에 재사용

스킬을 불러 작업  
필요한 조건만 추가

CLAUDE.md는 계속 적용할 규칙, SKILL.md는 해당 작업에서 사용할 절차입니다.  
지금 필요한 스킬을 고르고, 사용한 결과는 직접 확인합니다.

# 상관없는 일은 동시에

서브에이전트는 작업의 일부를 맡는 별도 AI 작업자입니다.

공통 자료: 회의 메모 + 매출 CSV

## 분석 담당

CSV의 수치·변화 분석

결과: 분석 요약

## 보고서 담당

메모로 보고서 초안 작성

결과: 배경·결정·할 일

서로 기다릴 필요가 없는 작업 나누기

서브에이전트 둘을 불러서 CSV 분석과 메모 기반 보고서 초안을 동시에 진행해줘.

끝나면 두 결과를 모아 보고서를 정리해줘.

보고서에 분석 수치가 필요하면, 분석이 끝난 뒤 반영합니다.

# 작업에 맞는 **추가 스킬** 선택

① 구현·디자인

② 검증·오류 분석

**필요한 부분만 구현하고 싶을 때**

**karpathy-guidelines**

가정을 먼저 확인하고

단순하게, 필요한 부분만 변경

[원문 · Karpathy의 관찰을 정리한 커뮤니티 스kill](#)

**화면에 개성을 주고 싶을 때**

**frontend-design**

사용 목적과 디자인 방향을 정해

글꼴·색·배치·상호작용 구현

[원문 · Anthropic](#)

원문을 읽고 지금 필요한 것 하나를 선택합니다.

# 작업에 맞는 **추가 스킬** 선택

① 구현·디자인

② **검증·오류 분석**

**동작을 브라우저에서 점검할 때**

**webapp-testing**

Playwright로 실제 조작하고  
화면·동작·오류 확인

[원문 · Anthropic](#)

실행 도구가 준비된 환경에서 사용

**고쳐도 같은 오류가 반복될 때**

**systematic-debugging**

오류를 재현하고 원인부터 찾은 뒤  
수정·다시 확인

[원문 · Superpowers](#)

원문을 읽고 지금 필요한 것 하나를 선택합니다.

# gstack: 역할별 개발 워크플로

① 역할·사용 시점

② 활용 예시

## 무엇을 돕나

기획 검토 · 설계 · 코드 리뷰 · QA

역할별 스킬이 작업을 이어받습니다.

하나의 스킬보다 개발 절차 묶음입니다.

## 언제 필요한가

앱의 방향부터 검증까지 정리할 때

기획: /plan-ceo-review

변경 검토: /review · 동작 점검: /qa

[제작자 원문 · gstack](#)

# gstack: 역할별 개발 워크플로

① 역할·사용 시점

② 활용 예시

설치·준비 후 사용할 예시

```
/plan-ceo-review
```

내 CSV 분석 서비스의 목적과 기능 범위를 검토해줘. 빠진 질문은 나에게 물어봐줘.

## 기대할 결과

핵심 문제 · 범위 · 빠진 가정

확인할 기획 문서와 다음 작업

## 내가 확인할 것

기획은 내가 검토하고 수정합니다.

리뷰와 실제 실행 검증을 함께 봅니다.

[제작자 원문 · gstack](#)

# taste-skill: 프론트엔드 디자인 기준

① 역할·사용 시점

② 활용 예시

## 무엇을 돕나

배치 · 글꼴 · 여백 · 움직임 기준  
반복적인 기본 화면을 개선합니다.  
레퍼런스와 목적을 구현에 반영합니다.

## 언제 필요한가

앱은 되지만 화면이 밋밋할 때  
레이아웃의 변화 정도와 움직임,  
한 화면의 정보량을 조절할 때

[제작자 원문 · taste-skill](#)

# taste-skill: 프론트엔드 디자인 기준

① 역할·사용 시점

② 활용 예시

설치·준비 후 사용할 예시

taste-skill을 활용해서 내 Work OS의 디자인 방향을 제안해줘.  
차분한 업무용 화면을 원해. 필요한 취향은 나에게 물어봐줘.

## 기대할 결과

레이아웃 · 색 · 글꼴 · 간격 기준  
기능을 담은 화면 디자인

## 내가 확인할 것

멋진 화면보다 읽기·조작도 확인합니다.  
움직임과 장식의 정도는 내가 정합니다.

[제작자 원문 · taste-skill](#)

# korean-privacy-terms: 개인정보 문서 초안

① 역할·사용 시점

② 활용 예시

## 무엇을 돕나

서비스에 필요한 정보를 인터뷰하고  
개인정보처리방침 · 이용약관 초안과  
동의 화면 구성을 돕습니다.

## 언제 필요한가

회원 정보나 개인정보를 받는 서비스  
수집 항목 · 목적 · 보관 기간을  
정리해 문서에 반영할 때

[제작자 원문 · korean-privacy-terms](#)

# korean-privacy-terms: 개인정보 문서 초안

① 역할·사용 시점

② 활용 예시

설치·준비 후 사용할 예시

/privacy-kr

내 서비스의 개인정보 처리방침 초안을 만들고 싶어. 필요한 정보를 나에게 물어봐줘.

## 기대할 결과

수집·이용·보관에 관한 문서 초안

서비스에 맞는 동의 항목

## 내가 확인할 것

실제 데이터 처리와 문서가 맞아야 합니다.

실서비스 문서는 법률 전문가가 검토합니다.

[제작자 원문 · korean-privacy-terms](#)

# ponytail: 필요한 만큼의 구현

① 역할·사용 시점

② 활용 예시

## 무엇을 돕나

요구사항과 기존 코드를 먼저 보고  
이미 있는 기능과 기본 도구를 활용해  
불필요한 코드·설치를 줄입니다.

## 언제 필요한가

작은 기능인데 코드가 크게 늘 때  
새 도구가 꼭 필요한지 고민될 때  
기존 기능으로 더 단순하게 풀 때

[제작자 원문 · ponytail](#)

# ponytail: 필요한 만큼의 구현

① 역할·사용 시점

② 활용 예시

설치·준비 후 사용할 예시

```
/ponytail
```

CSV 업로드 기능을 가장 단순한 방법으로 구현하고 싶어. 기존 기능을 활용할 방법부터 제안해줘.

## 기대할 결과

더 단순한 구현 방법과 변경 범위

기존 코드·브라우저 기능 활용

## 내가 확인할 것

저장 실패 처리와 보안은 유지합니다.

코드가 줄어도 같은 동작인지 확인합니다.

[제작자 원문 · ponytail](#)

# ui-ux-pro-max: UI/UX 설계 지식

① 역할·사용 시점

② 활용 예시

## 무엇을 돕나

서비스 종류에 맞는 디자인 지식을 찾아  
색 · 글꼴 · 배치 · 차트 · 사용 흐름을  
일관된 화면 규칙으로 정리합니다.

## 언제 필요한가

업무 화면의 기준이 필요할 때  
표·차트·입력 화면을 함께 설계할 때  
모바일 배치와 접근성을 점검할 때

[제작자 원문 · ui-ux-pro-max](#)

# ui-ux-pro-max: UI/UX 설계 지식

① 역할·사용 시점

② 활용 예시

설치·준비 후 사용할 예시

ui-ux-pro-max를 활용해서 내 CSV 분석 서비스의 화면 기준을 제안해줘.  
사용자와 주요 작업을 나에게 물어보고 색·글꼴·배치를 정리해줘.

## 기대할 결과

색·글꼴·간격·화면 구성 기준  
차트 선택과 UI/UX 확인 항목

## 내가 확인할 것

내 데이터와 사용 상황에 맞게 고릅니다.  
실제 폰 화면과 읽기 쉬운지도 확인합니다.

[제작자 원문 · ui-ux-pro-max](#)

# Understand-Anything: 코드 구조의 시각화

① 역할·사용 시점

② 활용 예시

## 무엇을 돕나

파일 · 함수 · 연결 관계를 분석해  
코드의 지식 그래프를 만듭니다.  
설명·검색·질문으로 구조를 살펴봅니다.

## 언제 필요한가

프로젝트가 커져 파일이 헛갈릴 때  
다른 사람이 만든 코드를 읽을 때  
어디를 바꾸면 될지 파악할 때

[제작자 원문 · Understand-Anything](#)

# Understand-Anything: 코드 구조의 시각화

① 역할·사용 시점

② 활용 예시

설치·준비 후 사용할 예시

/understand

이 프로젝트의 구조를 분석해줘. CSV 입력부터 분석 결과까지 흐름을 이해하고 싶어.

## 기대할 결과

파일·함수 연결 지도와 설명

/understand-dashboard로 구조 탐색

## 내가 확인할 것

지도와 실제 코드가 맞는지 확인합니다.

코드가 바뀌면 분석 결과도 갱신합니다.

[제작자 원문 · Understand-Anything](#)

# open-code-review: 변경 코드 점검

① 역할·사용 시점

② 활용 예시

## 무엇을 돕나

Git의 변경 내용과 관련 코드를 읽고  
오류 · 보안 · 성능 위험을 검토합니다.  
파일·줄 단위로 수정 의견을 제시합니다.

## 언제 필요한가

기능을 고친 뒤 공유하기 전  
AI가 만든 변경을 다시 살필 때  
고칠 문제의 우선순위를 정할 때

[제작자 원문 · open-code-review](#)

# open-code-review: 변경 코드 점검

① 역할·사용 시점

② 활용 예시

설치·준비 후 사용할 예시

open-code-review로 이번 변경의 오류와 보안 위험을 검토해줘.  
수정하기 전에 중요한 문제와 근거부터 알려줘.

## 기대할 결과

파일 위치 · 위험도 · 원인 · 수정 제안  
먼저 고칠 문제와 확인할 항목

## 내가 확인할 것

리뷰 의견을 읽고 반영 여부를 정합니다.  
수정 뒤 웹앱을 직접 실행해 확인합니다.

[제작자 원문 · open-code-review](#)

# 추가 기획 스킬의 전체 내용 설치

grill-me는 LV4에서 설치했습니다. 아래 두 스킬은 필요한 경우 추가합니다.

**brainstorming**

writing-plans

## 아이디어를 설계로 정리하는 스킬

목표를 확인하고 방법을 비교합니다.

화면·저장 방식의 초안을 함께 정합니다.

### LV7 · 웹앱 연결 설계

사용 예: /brainstorming으로 웹앱 연결을 함께 설계 해줘.

수업용 한국어판

원본: obra/superpowers · MIT

전체 SKILL.md + 저장 프롬프트

전체 SKILL.md와 저장 프롬프트는 다음 페이지부터 이어집니다.  
내용을 생략하지 않고 모두 실었습니다.

# 추가 기획 스킬의 전체 내용 설치 · 전체 내용 1

원본 슬라이드 87 · brainstorming · 이어지는 내용을 모두 제공합니다.

아래 내용을 `.claude/skills/brainstorming/SKILL.md`로 저장해줘.  
설치 후 파일 경로와 내용을 확인해줘.

```
```markdown
---
name: brainstorming
description: 기능을 만들거나 여러 조각을 합치기 전에, 사용자와 대화하며 의도를 확인하고 설계를 함께 정한다. 사용자가 /brainstorming 을 입력하거나 "같이 설계하자", "만들기 전에 정리하자"라고 할 때 쓴다.
---

# brainstorming – 만들기 전에 같이 설계하기

> 교육용 한국어판. 원본: Jesse Vincent `obra/superpowers`의 `brainstorming` (MIT License, 원문은 `_원본/obra-superpowers/`).
> 바꾼 점:
> – git 커밋·worktree, 테스트 먼저(TDD), 시각 도우미 브라우저, 다른 스킬 자동 호출을 뺐다.
> – 화면·로컬만 만드는 비개발자 수업에 맞춰 줄였다.
> – 핵심은 남겼다: **의도 확인 → 설계 → 승인 전에는 만들지 않기**.
```

# 추가 기획 스킬의 전체 내용 설치 · 전체 내용 2

원본 슬라이드 87 · brainstorming · 이어지는 내용을 모두 제공합니다.

아이디어를 사용자가 알아보고 고칠 수 있는 **\*\*설계\*\***로 바꾼다.

**## 꼭 지킬 것**

<HARD-GATE>

사용자가 설계를 승인하기 전에는 코드 작성, 파일 생성·수정, 설치를 하지 않는다.

폴더 안 파일을 **\*\*읽는 것\*\***은 해도 된다.

</HARD-GATE>

"너무 간단해서 승인까지 필요 없다"는 생각이 들어도 짧은 설계를 보여주고 승인을 받는다.

**## 순서**

1. **\*\*폴더 둘러보기\*\***: 관련 파일(기획.md, 기존 앱 폴더)을 읽고, 지금 무엇이 있는지 짧게 정리한다.
2. **\*\*의도 확인\*\***: 무엇을 이루려는지, 누가 쓰는지, 무엇이 되면 성공인지 확인한다.
  - 모르는 게 있으면 **\*\*한 번에 질문 하나\*\***만 한다.
  - 이미 답이 나와 있으면 다시 묻지 않는다.
3. **\*\*내가 이해한 것 써 보기\*\***: 목표·조건·성공 기준을 5줄 안쪽으로 쓴다.

# 추가 기획 스킬의 전체 내용 설치 · 전체 내용 3

원본 슬라이드 87 · brainstorming · 이어지는 내용을 모두 제공합니다.

- "사용자가 말한 것"과 "내가 가정한 것"을 나눠서 쓴다.
- 사용자의 정정을 받고 반영한다.
- 4. **\*\*방법 2~3개 비교\*\***: 장단점을 한 줄씩 쓰고 추천 하나를 고른다.
- 5. **\*\*설계 보여주기\*\***: 승인 전에 아래 항목을 쓴다.
  - 화면 구성(탭·카드·버튼)
  - 데이터를 어디에 어떻게 저장하나(오늘은 브라우저 저장)
  - 기존 조각에서 그대로 쓰는 것 / 바꾸는 것 / 버리는 것
  - 이번에 **\*\*안 하는 것\*\***
- 6. **\*\*승인 받기\*\***: "이 설계로 갈까요?"라고 묻는다.
  - 승인 범위를 넘겨짚지 않는다. 설계 승인은 "계획을 써도 된다"는 뜻이지 "만들어도 된다"는 뜻이 아니다.
- 7. **\*\*설계 문서 저장\*\***: 승인된 설계를 `docs/설계.md`에 저장한다.
  - 다음 단계는 `/writing-plans`로 계획을 쓰는 것이라고 안내한다.

## ## 하지 않는 것

- 승인 전에 만들기 시작하기
- 질문 여러 개를 한꺼번에 묻기
- 사용자가 말하지 않은 기능을 슬쩍 넣기

# 추가 기획 스킬의 전체 내용 설치 · 전체 내용 4

원본 슬라이드 87 · brainstorming · 이어지는 내용을 모두 제공합니다.

– 서버·데이터베이스·로그인 추가 – 오늘 수업 범위 밖이다. 사용자가 요구하면 "오늘 범위 밖"이라고 알린다  
` ` ` `

# 추가 기획 스킬의 전체 내용 설치

grill-me는 LV4에서 설치했습니다. 아래 두 스킬은 필요한 경우 추가합니다.

brainstorming

writing-plans

## 설계를 실행 단계로 나누는 스킬

승인한 설계를 작은 작업으로 나누고  
각 단계에서 확인할 결과를 적습니다.

### LV7 · 구현 순서

사용 예: /writing-plans로 승인한 설계를 단계로 나  
눠줘.

수업용 한국어판

원본: obra/superpowers · MIT

전체 SKILL.md + 저장 프롬프트

전체 SKILL.md와 저장 프롬프트는 다음 페이지부터 이어집니다.  
내용을 생략하지 않고 모두 실었습니다.

# 추가 기획 스킬의 전체 내용 설치 · 전체 내용 1

원본 슬라이드 87 · writing-plans · 이어지는 내용을 모두 제공합니다.

아래 내용을 `.claude/skills/writing-plans/SKILL.md`로 저장해줘.  
설치 후 파일 경로와 내용을 확인해줘.

```
```markdown
---
name: writing-plans
description: 승인된 설계를 작고 확인 가능한 단계의 실행 계획으로 쓴다. 사용자가 /writing-plans 를 입력하거나 "계획 써줘", "단계로 나눠줘"라고 할 때, 또는 brainstorming 설계가 승인된 뒤에 쓴다.
---

# writing-plans – 작은 단계로 계획 쓰기

> 교육용 한국어판. 원본: Jesse Vincent `obra/superpowers`의 `writing-plans` (MIT License, 원문은 `_원본/obra-superpowers/`).
> 바꾼 점:
> - git 커밋, worktree, 테스트 코드 먼저(TDD), 하위 에이전트 실행 안내를 뺐다.
> - 대신 단계마다 사람이 **localhost에서 놀러 보고 확인할 것**을 쓰게 했다.
```

# 추가 기획 스킬의 전체 내용 설치 · 전체 내용 2

원본 슬라이드 87 · writing-plans · 이어지는 내용을 모두 제공합니다.

이 프로젝트를 처음 보는 사람도 그대로 따라 할 수 있게 계획을 쓴다. 계획에는 **\*\*무엇을 정했는지\*\***를 남긴다: 어떤 파일, 어떤 이름, 설계의 어떤 값, 어떻게 확인할지.

## ## 시작 전

- `docs/설계.md` (brainstorming 결과)나 사용자가 준 설계를 읽는다.
- 설계가 없으면 먼저 `/brainstorming`을 권한다.
- 설계가 너무 크면 나눠서 계획하자고 제안한다.

## ## 계획 문서 형식

`docs/계획.md`에 저장한다.

```markdown

# [기능 이름] 실행 계획

**\*\*목표:\*\*** 한 문장

**\*\*설계 문서:\*\*** docs/설계.md

# 추가 기획 스킬의 전체 내용 설치 · 전체 내용 3

원본 슬라이드 87 · writing-plans · 이어지는 내용을 모두 제공합니다.

**\*\*지킬 것:\*\*** 서버·DB 없음, 브라우저 저장만, 기획.md에 없는 기능 넣지 않기

## 파일 지도

- `index.html` - 무엇을 담당
- `app.js` - 무엇을 담당

## 단계 1: [이름]

- [ ] 할 일: 무엇을 어떤 파일에서 바꾸나 (구체적으로)
- [ ] 확인: localhost에서 무엇을 누르면 무엇이 보여야 하나
- [ ] 멈춤: 사용자 확인을 받고 다음 단계로

## 단계 2: ...

``

## 단계 크기

- 단계는 **\*\*3~5개\*\***로 한다. 사용자가 개수를 정하면 따른다.
- 한 단계는 **\*\*눌러서 확인할 수 있는 변화 하나\*\***만 담는다.

# 추가 기획 스킬의 전체 내용 설치 · 전체 내용 4

원본 슬라이드 87 · writing-plans · 이어지는 내용을 모두 제공합니다.

- 단계마다 "확인"을 반드시 쓴다. 확인할 수 없으면 너무 크거나 모호한 것이다.

## ## 스스로 점검 (저장 전에)

- 설계의 모든 항목이 어느 단계에 들어갔나?
- "알아서 적당히", "필요하면" 같은 빈 말이 남아 있지 않나?
- 단계 사이의 이름(파일명·버튼 이름)이 서로 맞나?

## ## 저장 후

계획 위치를 알리고 묻는다. "단계 1부터 시작할까요? 단계마다 멈춰서 확인받겠습니다."

\*\*사용자가 시작하라고 하기 전에는 만들지 않는다.\*\*

````

# 반복할 작업을 위한 내 스킬 제작

자주 쓰는 작업 방법을 SKILL.md로 저장합니다.

① AI와 초안 만들기

② 파일 구조·사용

인터뷰

언제·무엇을 할지



초안 확인

순서·확인 기준



저장·사용·수정

실제 결과로 개선

예시 · 웹앱 점검 스킬 만들기

웹앱을 수정할 때마다 같은 방법으로 점검하고 싶어.

나를 인터뷰해서 점검 순서와 기준을 정하고 SKILL.md 초안을 만들어줘.

내가 확인하면 `.claude/skills/webapp-check/SKILL.md`로 저장해줘.

# 반복할 작업을 위한 내 스킬 제작

자주 쓰는 작업 방법을 SKILL.md로 저장합니다.

① AI와 초안 만들기

② 파일 구조·사용

SKILL.md · 파일에 넣는 내용 예시

---

**name:** webapp-check

**description:** 웹앱을 수정한 뒤 버튼 동작과 저장 유지 점검

---

1. 변경한 기능의 기대 결과를 확인한다.
2. 브라우저에서 직접 조작하고 저장·새로고침을 확인한다.
3. 통과·실패·미확인을 보고하고 수정할 부분을 제안한다.

## 파일에 쓰는 세 가지

**name** · 호출할 이름

**description** · 언제 쓰는지

**본문** · 실제 작업 순서

저장: .claude/skills/webapp-check/SKILL.md

**/webapp-check**로 사용

결과를 보고 내용 수정

# 사진 기록 웹앱의 실행 예시

직접 만든 수업용 예시 · 사진 선택 → 메모 → 저장

① 사진 선택 전

② 기록 저장 후

수업용 예시

## 내 사진 기록

사진과 한 줄 메모로 남기는 내 기록

사진 선택

파일 선택

선택된 파일 없음

미리보기

한 줄 메모

오늘의 한 줄을 적어보세요

저장

아직 저장한 기록이 없어요. 왼쪽에서 첫 기록을 남겨보세요!

사진과 메모를 입력하는 화면

# 사진 기록 웹앱의 실행 예시

직접 만든 수업용 예시 · 사진 선택 → 메모 → 저장

① 사진 선택 전

② 기록 저장 후

## 내 사진 기록

사진과 한 줄 메모로 남기는 내 기록

사진 선택

파일 선택 선택된 파일 없음

미리보기

한 줄 메모

오늘의 한 줄을 적어보세요

저장



오르르

2026년 10월 8일

선택한 이미지와 메모가 카드로 표시된 화면

# 사진 분석의 API 확장

① 분석 기능 기획

② 개발·확인

③ 날씨와 결합

## 사진에서 무엇을 알고 싶나요?

내 사진의 내용 설명

취미 기록의 특징 정리

목적과 결과 화면을 함께 결정

### Plan · 사진 분석 기획

이 사진 기록 앱에 Claude API로 사진을 분석하는 기능을 더 하고 싶어.

/grill-me로 나를 인터뷰해서 분석 목적과 화면을 함께 기획하자.

# 사진 분석의 API 확장

① 분석 기능 기획

② 개발·확인

③ 날씨와 결합

기획 확인 후 · 단계별 개발

확인한 기획대로 사진 분석 기능을 개발하자.

개발 단계를 나누고 첫 단계부터 구현해줘. 내가 확인하면 다음으로 넘어가자.

## 직접 사용

사진 선택 → 분석 → 결과 읽기

사진과 설명이 맞는지 비교

## 결과 확인·수정

추측한 내용은 확인하고 수정

내가 확인한 결과를 기록에 저장

# 사진 분석의 API 확장

① 분석 기능 기획

② 개발·확인

③ 날씨와 결합

서로 다른 API의 결과를 모아 사진에 어울리는 설명을 만들 수 있습니다.

## 사진 분석

Claude가 읽은 사진 내용

+

## 날씨 정보

도시 · 날씨 · 데이터 시각



## 설명 문구

Claude가 내용과 날씨를 연결

사진 분석과 날씨 조회가 각각 동작한 뒤, 두 결과를 어떻게 연결할지 함께 정합니다.

## 설명용 예시

공원 사진 + 맑음·20°C → “맑은 날 공원에서 남긴 오늘의 기록”

# 사진 기록에 더하는 날씨 API

도시를 고르면 날씨 API의 온도를 사진 기록에 함께 남깁니다.

## Open-Meteo · 무료 교육용 체험

도시 선택

seoul

날씨 가져오기

버튼을 누르면 실제 날씨 API를 호출합니다.

날씨 출처: [Open-Meteo](#)

사진 메모에 날씨 추가 · 기획 프롬프트

사진 메모에 내가 고른 도시의 날씨도 남기고 싶어.

Open-Meteo API로 온도를 가져오고 싶어.

나를 인터뷰해서 필요한 기능을 정리하고 IA를 수정해줘.

IA를 내가 승인하면 날씨 조회·저장·실패 시 동작을 더 물어 PRD를 구체화하자.

내가 PRD를 검토·승인하면 구현 계획을 세우자.

# 스마트폰 시연의 터널 링크

AI 에이전트에게 말로 웹앱을 만드는 것이 바이브코딩입니다.  
터널링 링크는 로컬 서버를 인터넷 주소에 연결해,  
외부 PC나 모바일에서 그 웹앱을 바로 열고 조작하게 합니다.



강사 시연·선택 · 스마트폰 연결

이 웹앱을 스마트폰에서 열 수 있게 터널링 링크를 만들어줘.

# 분석 결과의 할 일 분해

분석으로 끝내지 않고, 내가 실행할 수 있는 작업으로 나눕니다.

## 입력 · 수업용 예시

**CSV:** A 100톤 · B 150톤

**메모:** 금요일 회의 전

판매 흐름을 확인하고 싶다

## AI가 제안할 작은 작업

1. 지역별 판매량과 총합 확인
2. 차이가 큰 항목과 근거 정리
3. 회의에서 확인할 질문 준비

## 내가 확인하고 수정할 것

할 일 · 우선순위 · 담당 · 기한 · 완료 상태

정해지지 않은 담당과 기한은 AI가 묻거나 '미정'으로 표시

# Phase별 단계적 개발

Phase는 개발을 나눈 단계입니다.

기획 확인 후 개발 프롬프트

확인한 기획을 개발 Phase별로 나눠줘.

각 Phase의 목표와 확인할 결과를 짧게 정리해줘.

첫 Phase부터 구현하고 내가 확인하면 다음으로 넘어가줘.

한 단계씩 확인하면 문제가 생긴 지점을 찾기 쉽습니다.

# 내 메모를 반영한 보고서 작성

보고서에 담고 싶은 내용을 말하고, AI와 프롬프트를 정합니다.

이렇게 물어보기

내 메모를 분석해서 보고서를 쓰려면 프롬프트를 어떻게 설정하면 좋을까?  
더 필요한 정보가 있으면 나에게 물어봐줘.  
내 답을 바탕으로 보고서 작성 프롬프트를 만들어줘.

# Playwright: 브라우저 점검 자동화

Microsoft가 개발한 웹 브라우저 자동화·테스트 프레임워크입니다.

코드로 브라우저를 제어해 사람이 하던 웹 작업을 자동화합니다.

## 자동으로 할 수 있는 웹 작업

페이지 이동 · 버튼 클릭 · 폼 입력

페이지의 데이터 읽기·수집

저장 → 새로고침 → 기록 유지 확인

## 나중에 활용할 때

같은 동작을 반복 점검할 때

기능 수정 후 기존 동작을 확인할 때

나중에 AI에게 “Playwright로 이 웹앱의 주요 기능을 점검해줘”라고 말해 보세요.

출처: [Microsoft Playwright](https://playwright.dev)

# 오늘 배운 것을 **내 말로** 점검하기

오늘 만든 웹앱을 보며 설명하고 직접 눌러 봅니다.

## ① 원하는 것을 정했나요?

누구의 어떤 문제인가?  
화면 IA와 조건 PRD는 어떻게 다른가?  
AI의 제안을 읽고 고친 부분은?

## ② 만들고 고칠 수 있나요?

파일을 지정해 Plan으로 계획하기  
확인한 계획을 구현하기  
로컬 주소로 웹앱 열기

## ③ 결과를 확인했나요?

내가 원한 화면·동작과 비교하기  
입력 → 저장 → 새로고침 해 보기  
다음에 고칠 한 가지 말하기

설명하기 어려운 부분은 교안으로 돌아가 내 프로젝트에서 다시 확인합니다.

# 작은 해결 방법을 주변인에게 써 보게 하기

로컬에서 확인한 웹앱을 배포하고, 실제 사용자의 반응으로 고칩니다.

## 내 PC에서 직접 확인

기획과 화면·동작 비교

변경 내용을 남기기

### Git · GitHub

수정 이력과 프로젝트 보관

접속할 주소 만들기

### Vercel 등 호스팅

다른 사람이 링크로 열기

AI API를 함께 쓴다면

### 서버에서 호출

API 키와 사용량 관리

같은 기록을 공유한다면

### 데이터 저장소

저장·접근 권한 정하기

처음에는 작은 기능 하나를 배포합니다. 공유 전 개인정보·API 키·공개 범위를 확인합니다.

# 앞으로 2주, 문제에서 배포까지

내 주변의 불편 하나를 골라, 실제 주변인이 써 보는 작은 해결 방법을 만듭니다.

## 1주차 · 문제 찾고 기획하기

- 내 주변에서 반복되는 불편을 찾아보기
- 겪는 사람에게 언제·어떻게 불편한지 인터뷰하기
- 인터뷰 내용을 AI와 정리하고, 나도 질문에 답하기
- 작은 솔루션의 IA → PRD를 읽고 수정·승인하기

## 2주차 · 만들고 배포해 보기

- 승인한 계획으로 핵심 기능 하나 구현하기
- 직접 눌러 보고 기대와 실제의 차이 고치기
- 공유 가능한 주소로 배포하기
- 처음 인터뷰한 사람에게 써 보게 하고 다시 고치기

남길 것: 발견한 문제 · 실제 인터뷰 메모 · 기획 문서 · 배포 링크 · 사용자 피드백과 다음 수정